KENNESAW STATE
UNIVERSITY

CS 4277
Neural Network

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Learning Objectives

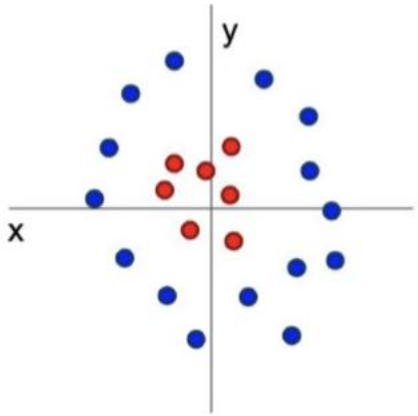
- Neural Network Introduction
- Activation Functions
- Multilayer Neural network

Linear Classifiers are not very powerful

- Visual viewpoint

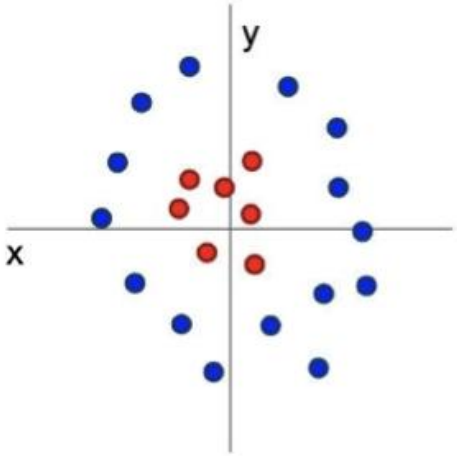
Linear classifiers learn one template per class

- Geometric Viewpoint



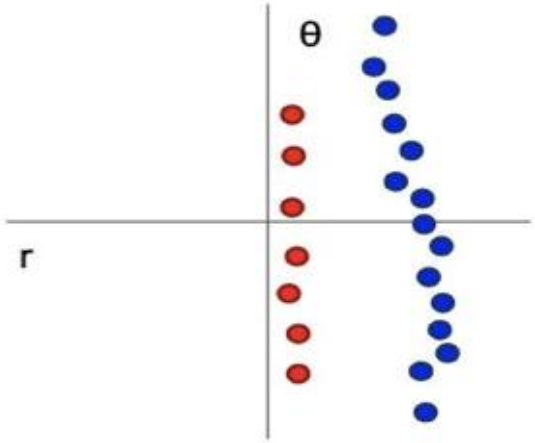
Linear classifiers can only draw linear decision boundaries

One Solution: Feature Transformation

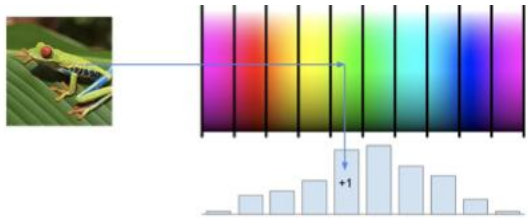


$$f(x, y) = (r(x, y), \theta(x, y))$$

Transform data with a cleverly chosen feature transform f, then apply linear classifier



Color Histogram



Histogram of Oriented Gradients (HoG)

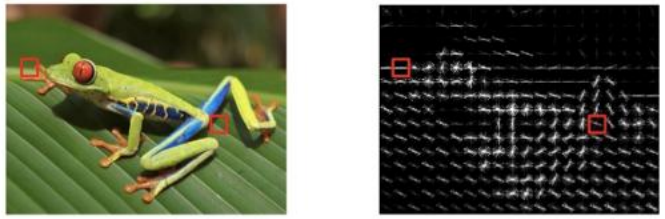
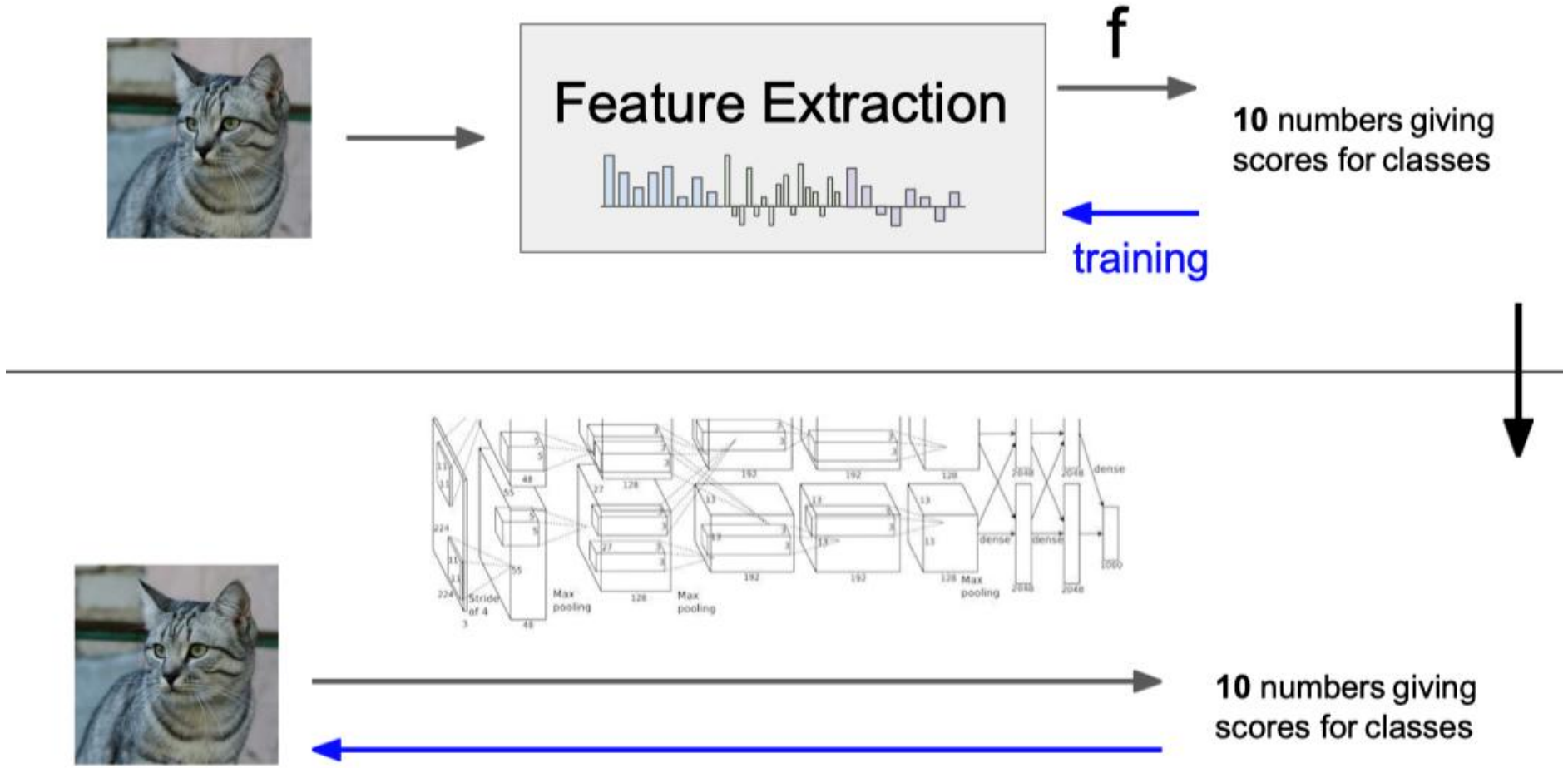
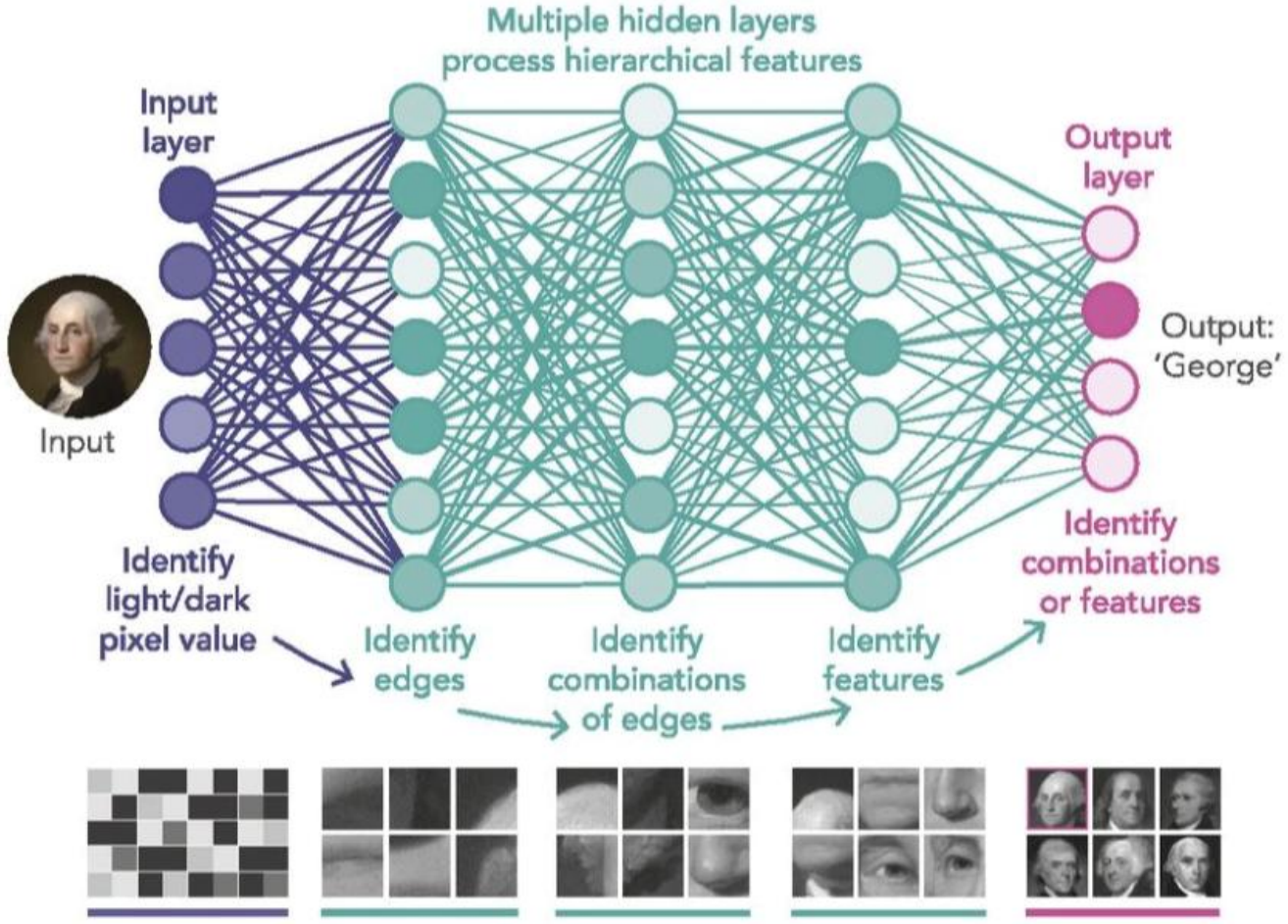


Image features vs Neural Networks



Deep Learning



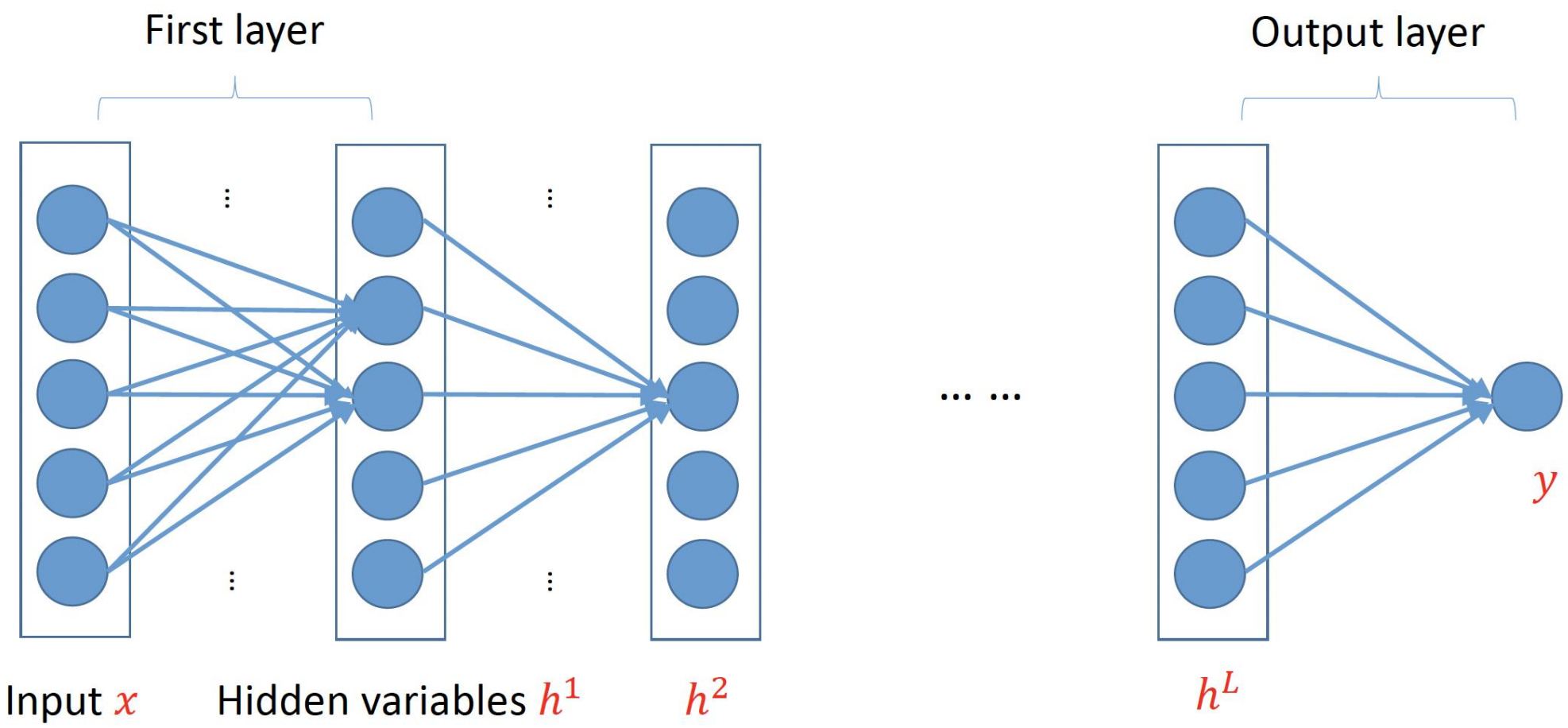
Mitchell Waldrop, "News Feature: What are the limits of deep learning?"

Feedforward networks

Components in Feedforward networks

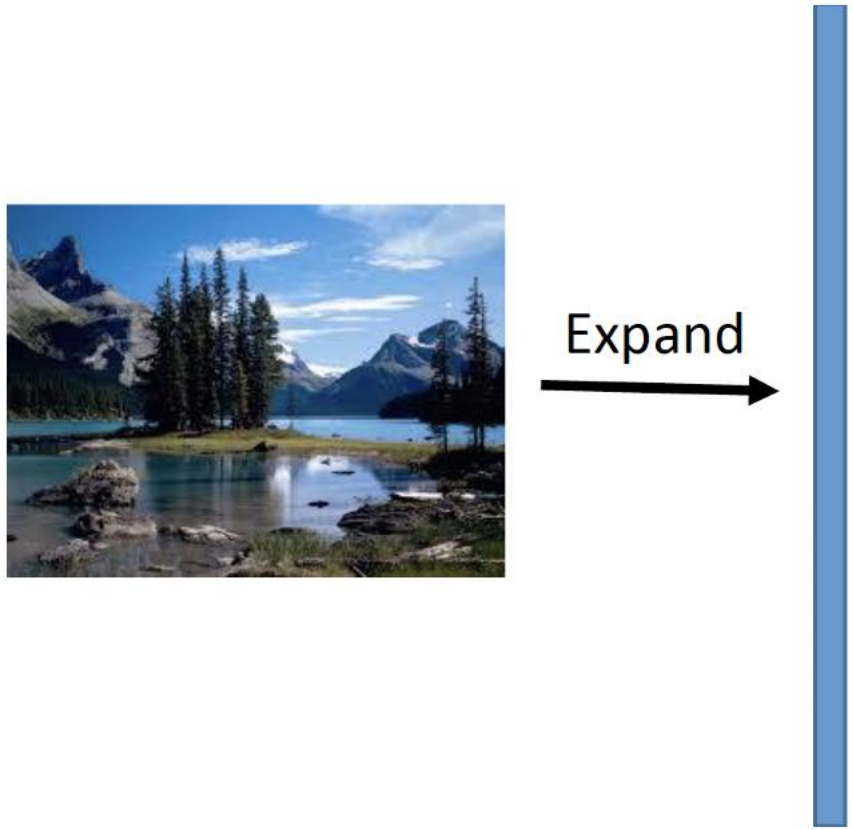
- Components
  - Representations:
    - Input
    - Hidden variables
  - Layers/weights:
    - Hidden layers
    - Output layer

Components



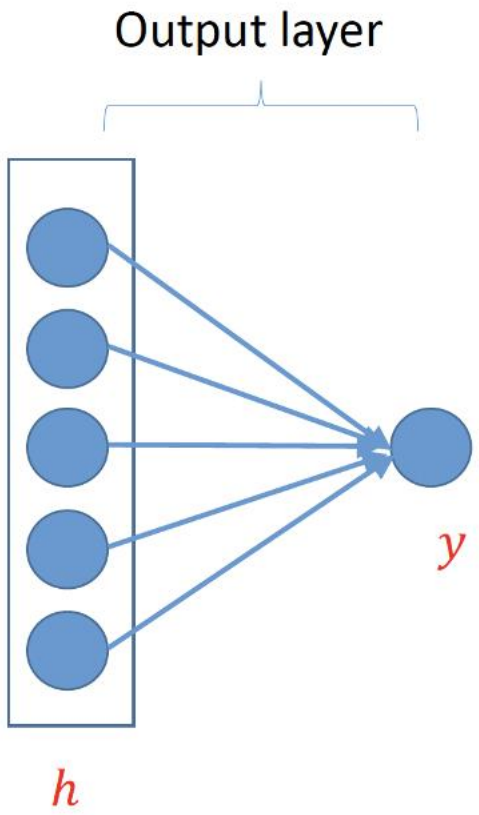
Input

- Represented as a vector
- Sometimes require some preprocessing
  - Subtract mean
  - Normalize to [-1, 1]
  - Normalize to [0, 1]



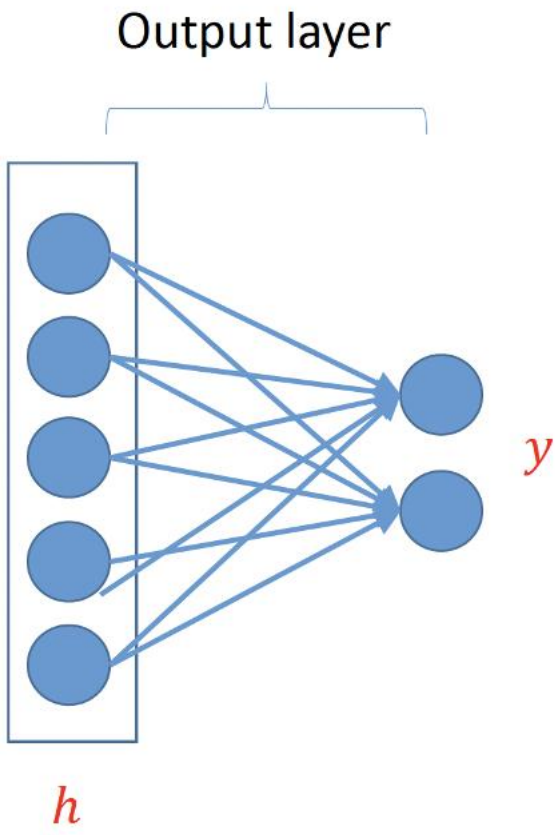
Output layer

- Regression: $y = w^T h + b$
- Linear units: no nonlinearity



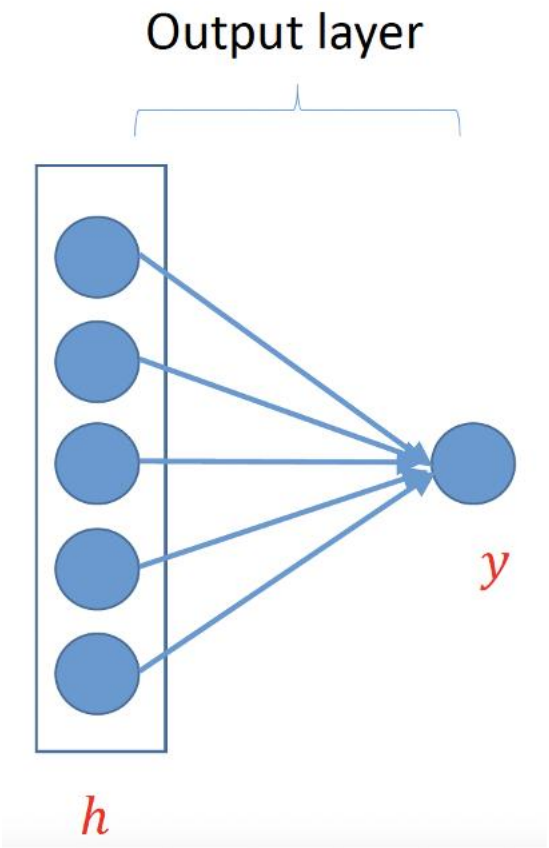
Output layer

- Multi-dimensional regression: $y = W^T h + b$
- W is a weight matrix
- Linear units: no nonlinearity



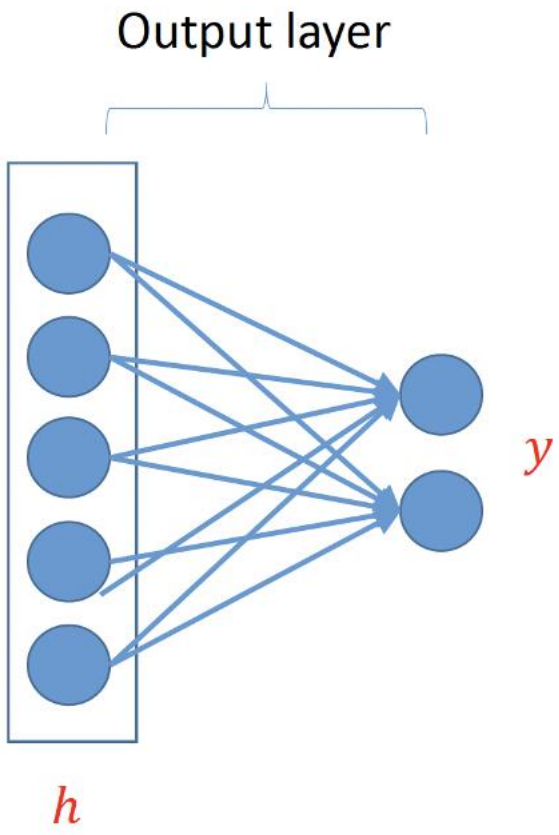
Output layer

- Binary classification $y = \sigma(w^T h + b)$
- Logistic regression



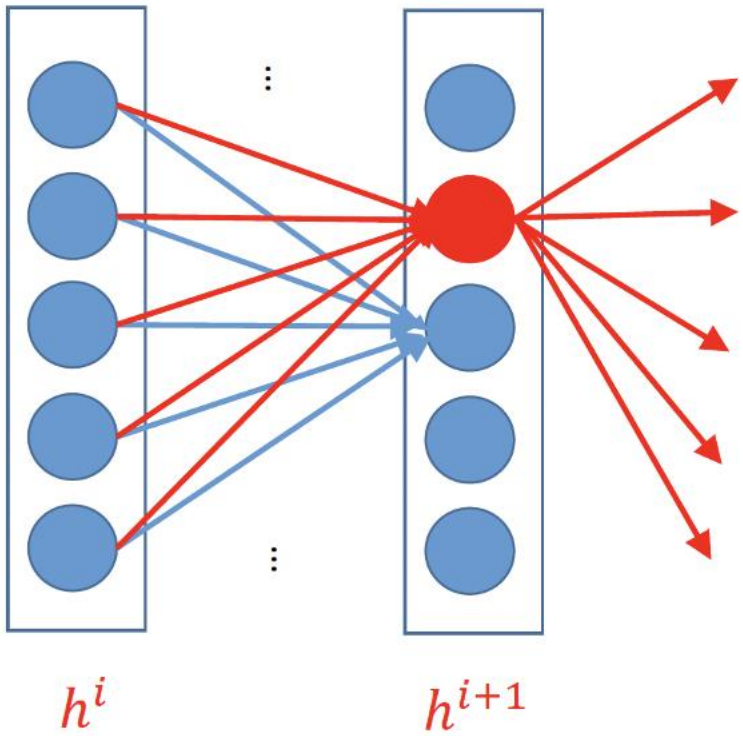
Output layer

- Multi-dimensional regression:
- $y = \mathrm{softmax}(W^T h + b)$
- W is a weight matrix
- Linear units: no nonlinearity



Hidden layer

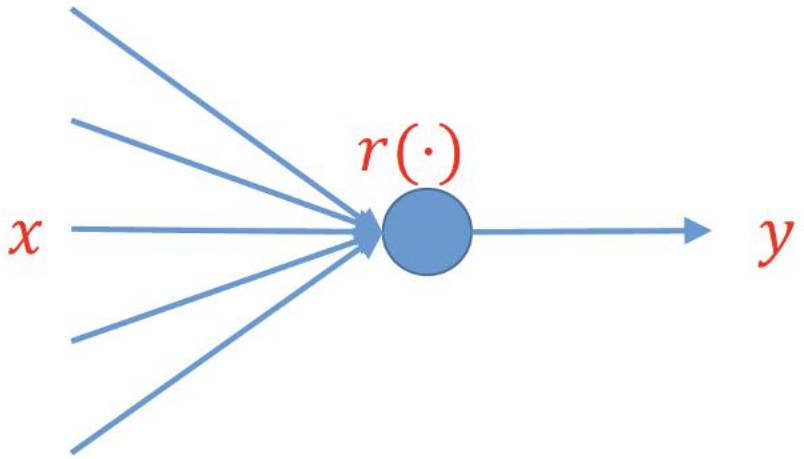
- Neuron take weighted linear combination of the previous layer
- Outputting one value for the next layer



Hidden layer

- $y = r(w^T h + b)$

- Typical activation function r
  - Threshold
    $$t(z) = \mathbb{I}[z \geq 0]$$
  - Sigmoid
    $$\sigma(z) = \frac{1}{1 + e^{-z}}$$
  - Tanh
    $$\tanh(z) = 2\sigma(2z) - 1$$

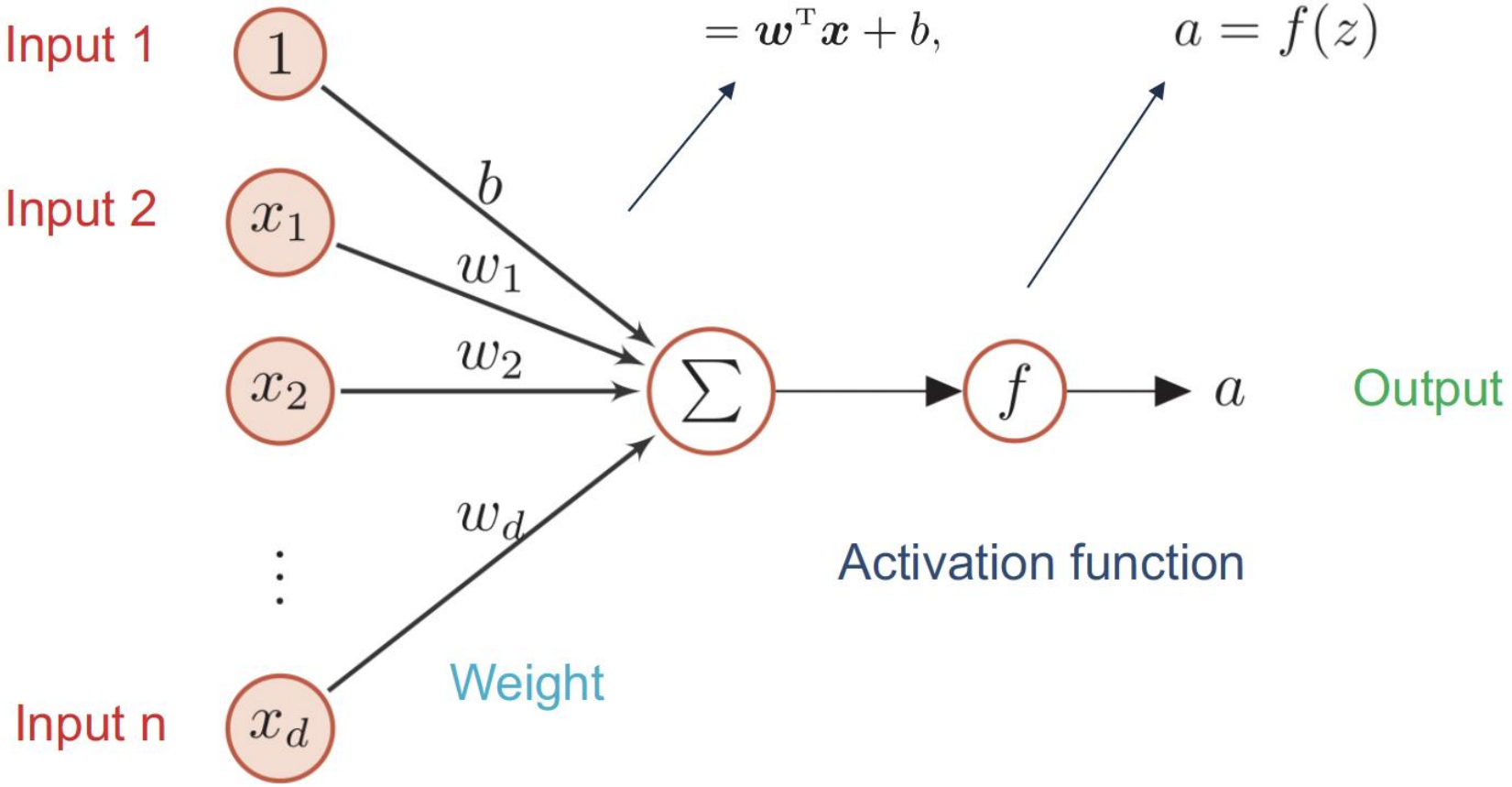


Computation unit

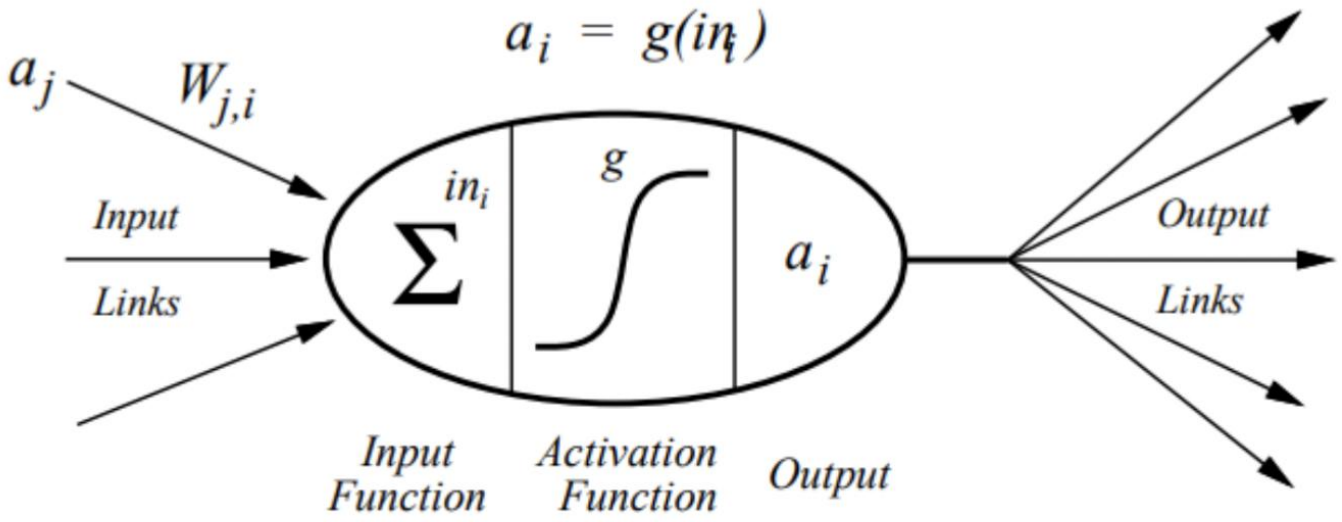
$$z = \sum_{i=1}^{d} w_i x_i + b$$

$$= \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + b,$$

$$a = f(z)$$



Artificial neuron (Simplified)



$$a_i = g(\sum_j W_{j,i} a_j)$$

Activation functions

Motivation

- Introducing Non-Linearity:
  - Without non-linear activation functions, the entire neural network would behave like a linear model, and the stack of linear layers would be equivalent to a single linear transformation.
  - The non-linear transformations allow the network to model and understand more sophisticated structures.
- Enabling Representation of Abstractions:
  - Each activation function processes the input and contributes to the creation of complex representations.
  - This hierarchical representation allows the network to learn hierarchical features, capturing increasingly abstract concepts.
- Handling Variability and Non-Linear Patterns:
  - Real-world data often contains non-linear patterns and variations.

The nature of the activation function

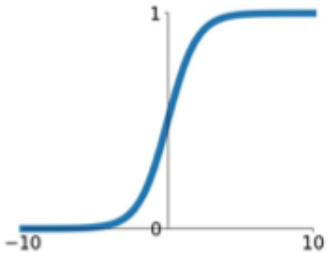
- Continuous and differentiable (allowing a few points to be non-differentiable) nonlinear function.
  - The derivable activation function can directly use numerical optimization methods to learn network parameters.
- The activation function and its derivative function should be as simple as possible
  - Conducive to improving network computing efficiency.
- The value range of the derivative function of the activation function should be in a suitable interval
  - It cannot be too large or too small, otherwise it will affect the efficiency and stability of training.
- Monotonically increasing

Monotonically increasing

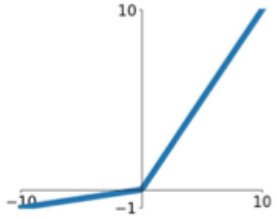
- Enable Learning Through Backpropagation:
  - Gradient descent minimizes the error or loss function. For effective optimization, it is crucial that the gradient of the activation function can guide the learning process. If the activation function is not monotonically increasing, the gradient may not consistently point in the direction of minimizing the error, making it challenging for optimization algorithms to converge.
- Preserve Information Flow:
  - An increase in the input to a neuron results in an increase in the output.
- Avoid Saturation (escape from local minimum or maximum):
  - Monotonically increasing activation functions help mitigate the risk of saturation.
- Support Positive and Negative Inputs:
  - Monotonically increasing functions are defined across the entire real number line. This allows activation functions to handle both positive and negative inputs, providing flexibility in representing various patterns and relationships in the data.

Common Activation Functions

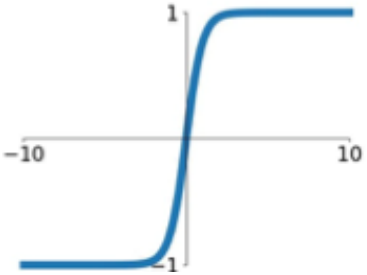
Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$



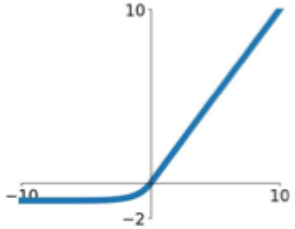
Leaky ReLU $\max(0.1x, x)$



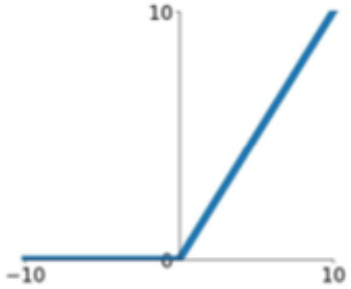
Tanh(x) $\frac{e^x - e^{-x}}{e^x + e^{-x}}$



ELU $\begin{cases} x, if\ x \geq 0 \\ \alpha(e^x - 1), if\ x < 0 \end{cases}$



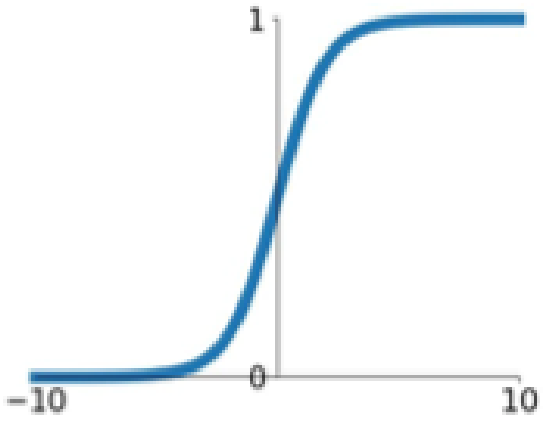
ReLU $\max(0, x)$



ReLU is a good default choice for most problems

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



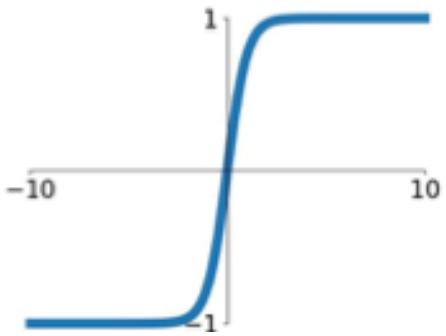
- S-shaped
- Non-linear function
- Output range [0,1]
- Linear near 0
- Outlier values squashed towards 0 or 1
- Function is differentiable
- function is monotonic but function's derivative is not. $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$
- Non zero Centered Activation Function
- The model Learning rate is slow
- May get stuck at the training time.
  - Create a Vanishing gradient problem.

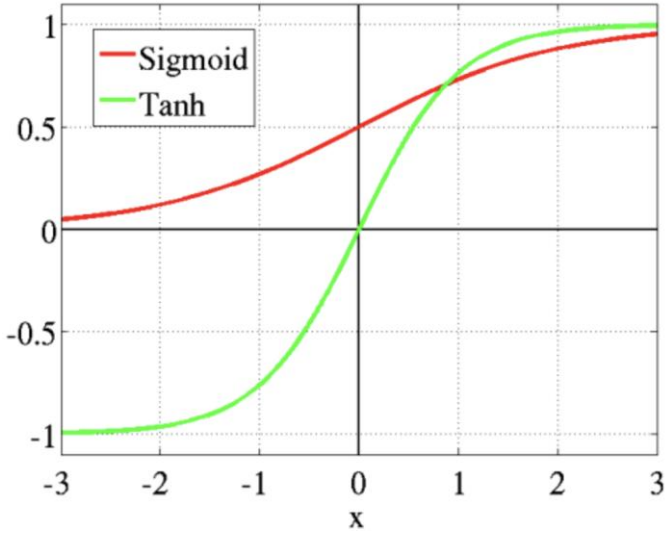
In the sigmoid or tanh functions, very high saturated values of z result in values of y that are saturated, i.e., extremely close to 1, and have derivatives very close to 0. Zero derivatives cause problems for learning.

Tanh

$$\tanh \quad \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

- Hyperbolic tangent activation function
- Range of tanh function is [-1, 1]
- S-shaped
- Non-linear
- Negative input mapped strongly negative
- Zero input mapped near zero in the graph
- Differentiable
- **zero centered activation function.**
- function is **monotonic** while its **derivative is not monotonic**.
- Vanishing Gradient Problem like a sigmoid function

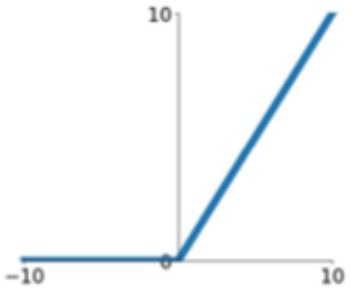




Rectified linear unit (ReLU)

- Linear for positive values
- Nonlinear for negative values
- function and its derivative **both are monotonic**.
- Ranges [0, infinity)
- Convert negative values to zero immediately
- Removed Vanishing Gradient Problem completely
  - Maximum Threshold values are Infinity
- Speed is fast compared to other activation function

ReLU $\max(0, x)$



ReLU and related functions

$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$= \max(0, x).$$

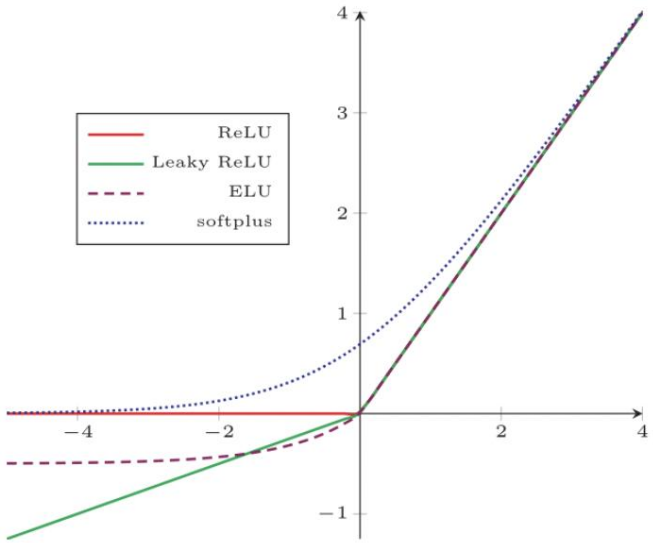
$$\mathrm{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

$$\mathrm{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\mathrm{softplus}(x) = \log(1 + \exp(x))$$

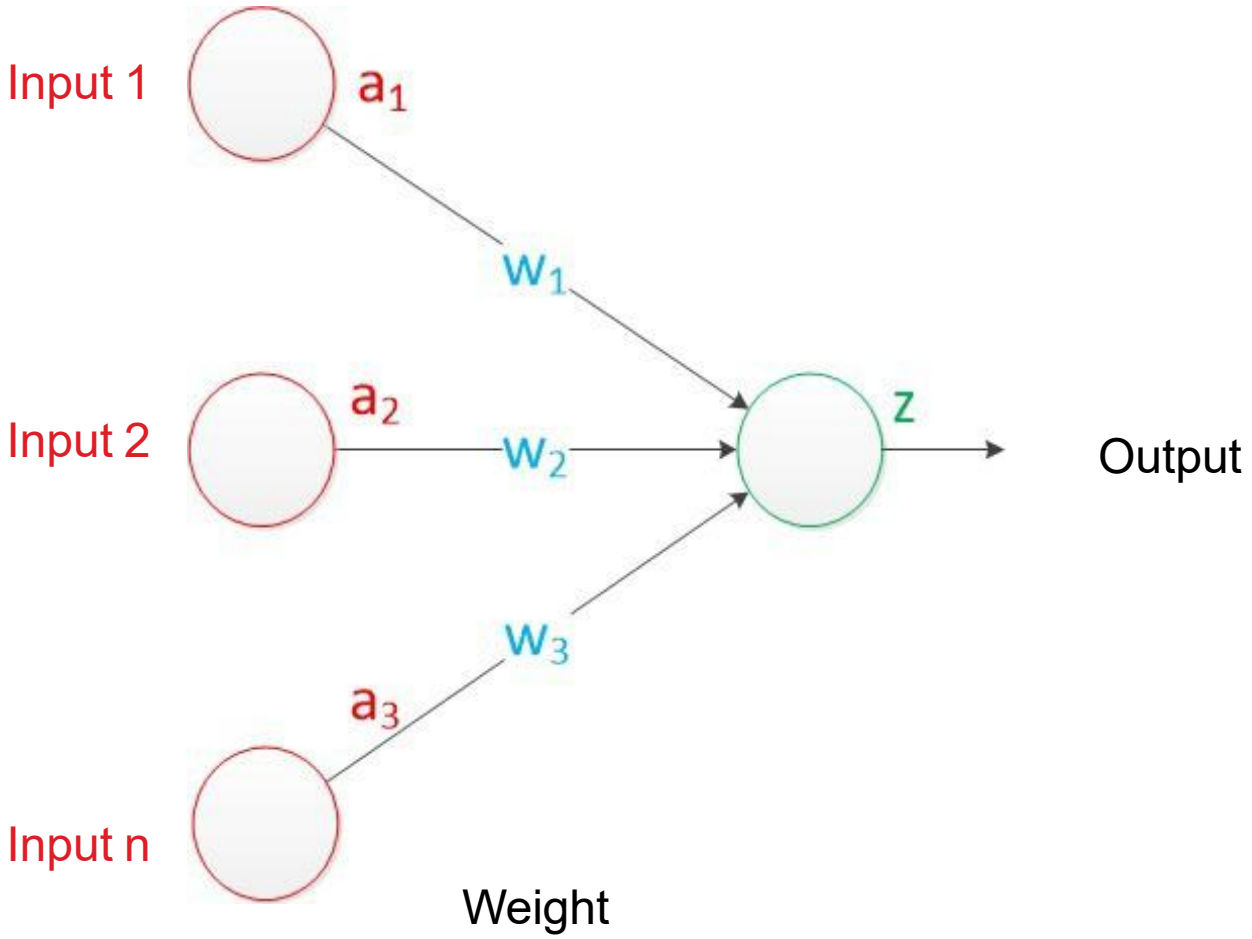


Dying ReLU Problem

- Computationally more efficient
- Biological rationality
- Unilateral inhibition, wide excitability boundary
- To a certain extent alleviate the problem of vanishing gradient

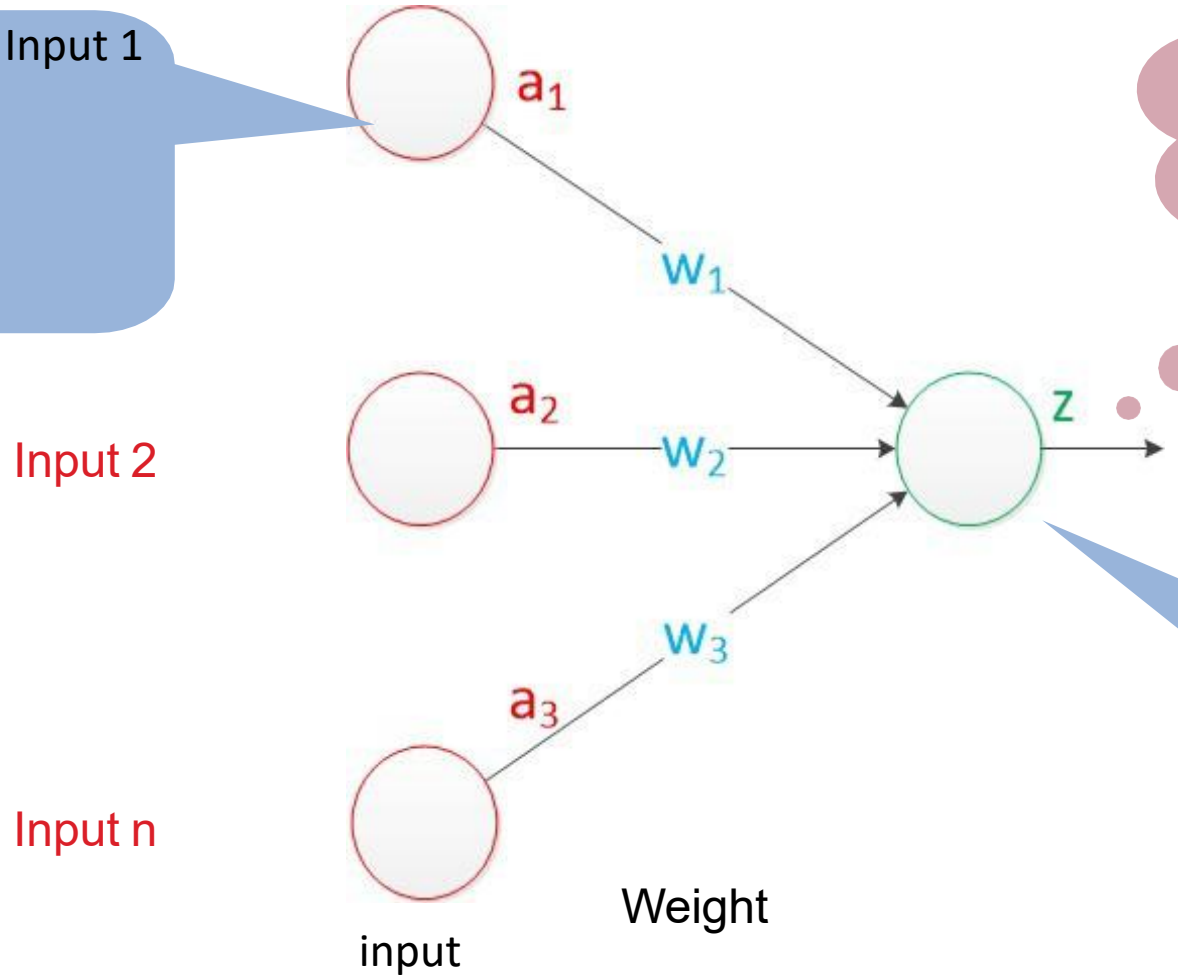
Single Layer Neural Network

Single layer neural network (Perceptron)



Single layer neural network (Perceptron)

The "input unit" in the input layer is only responsible for transmitting data not calculating

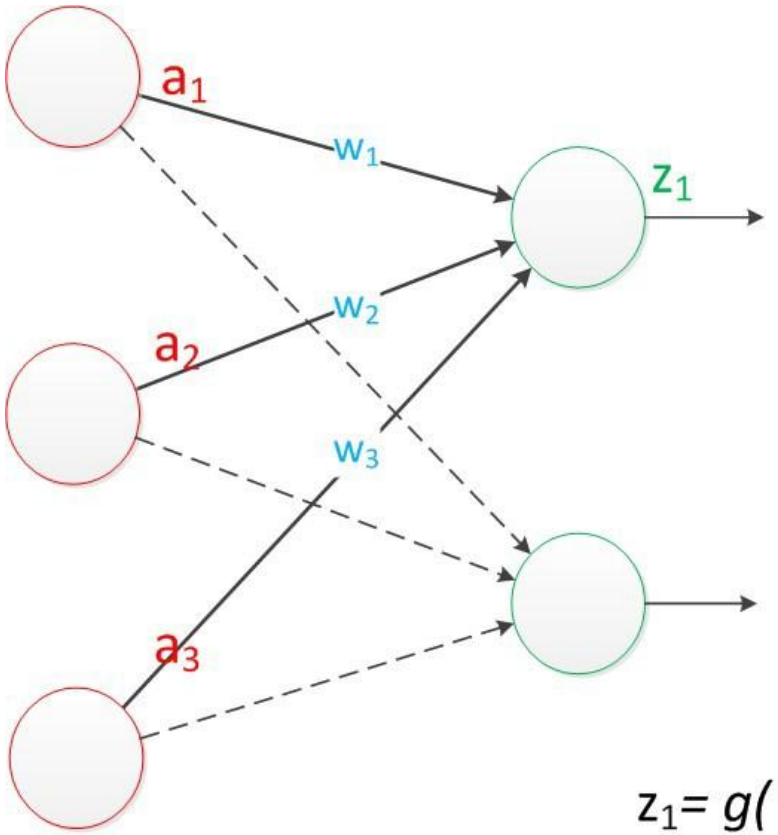


The layer needs to be calculated is called the " computing layer"

The "output unit" in the output layer needs to calculate the input of the previous layer

Single layer neural network (Perceptron)

- A network with one computing layer is called a "single layer neural network"

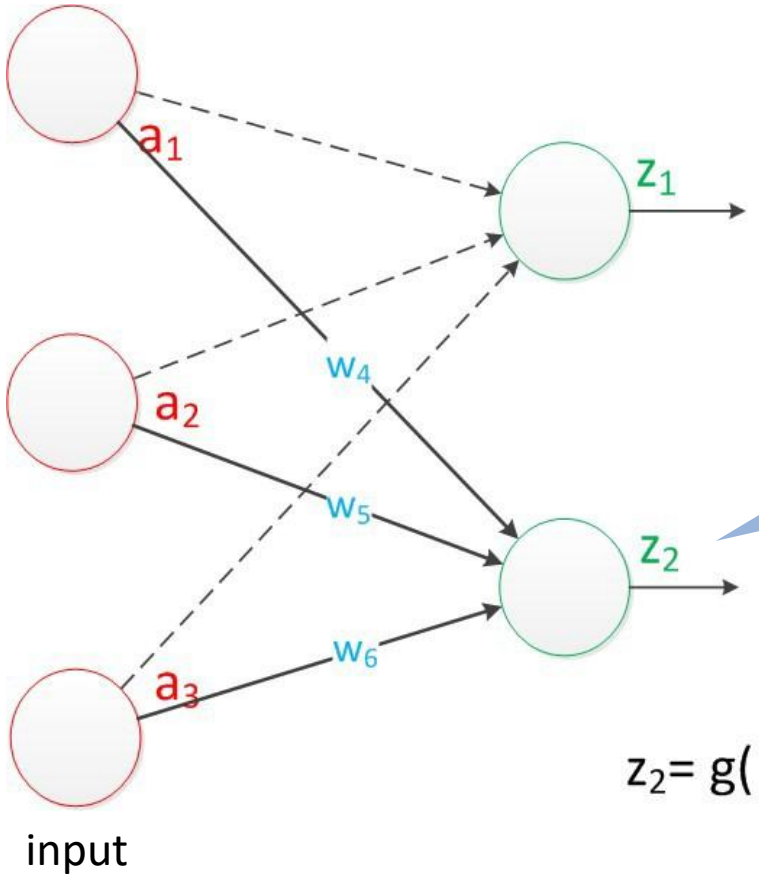


input

$$z_1 = g(a_1 * w_1 + a_2 * w_2 + a_3 * w_3)$$

Single layer neural network (Perceptron)

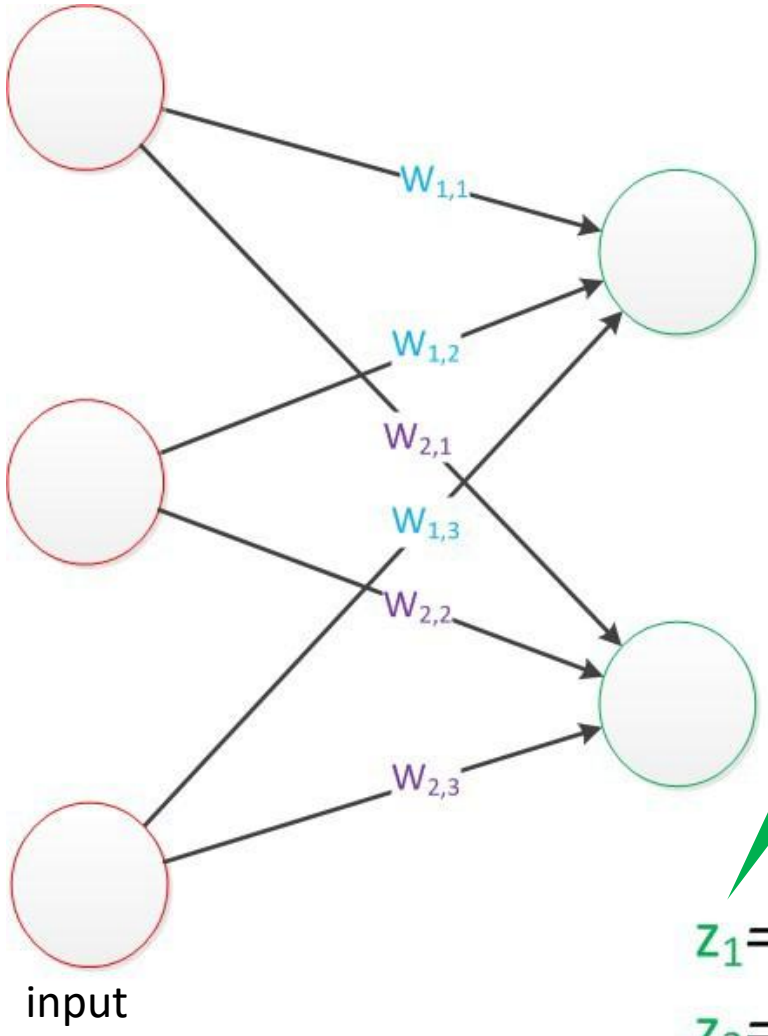
- A network with one computing layer is called a "single layer neural network"



The calculation of z_2 is the same as z_1 except for three new weights: w_4, w_5, and w_6

$$z_2 = g(a_1 * w_4 + a_2 * w_5 + a_3 * w_6)$$

Single layer neural network (Perceptron)



$$g(\mathbf{W} * \mathbf{a}) = \mathbf{z}$$

$z = [z_1, z_2]^T$

$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}$

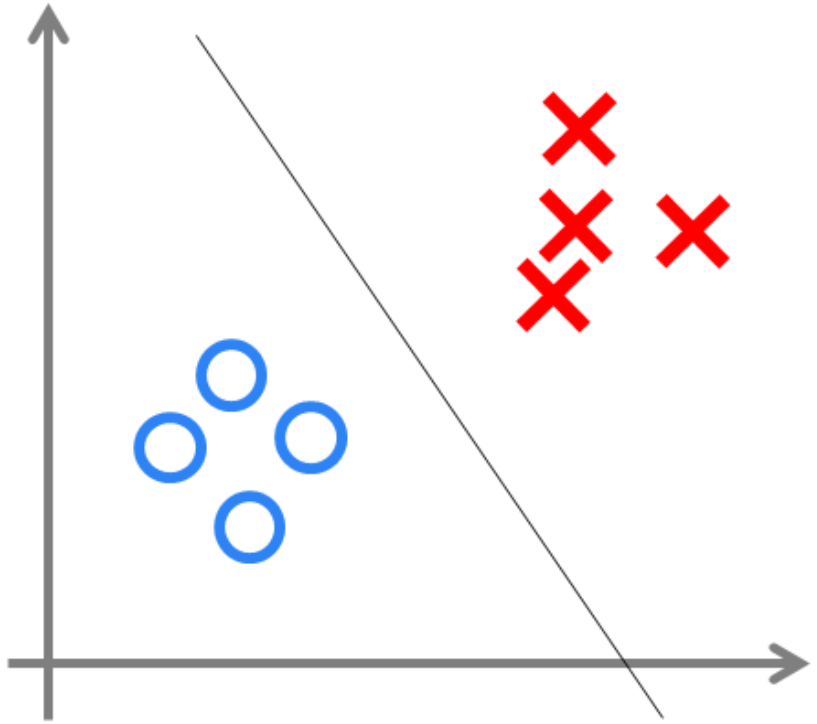
$a = [a_1, a_2, a_3]^T$

$$z_1 = g(a_1 * w_{1,1} + a_2 * w_{1,2} + a_3 * w_{1,3})$$

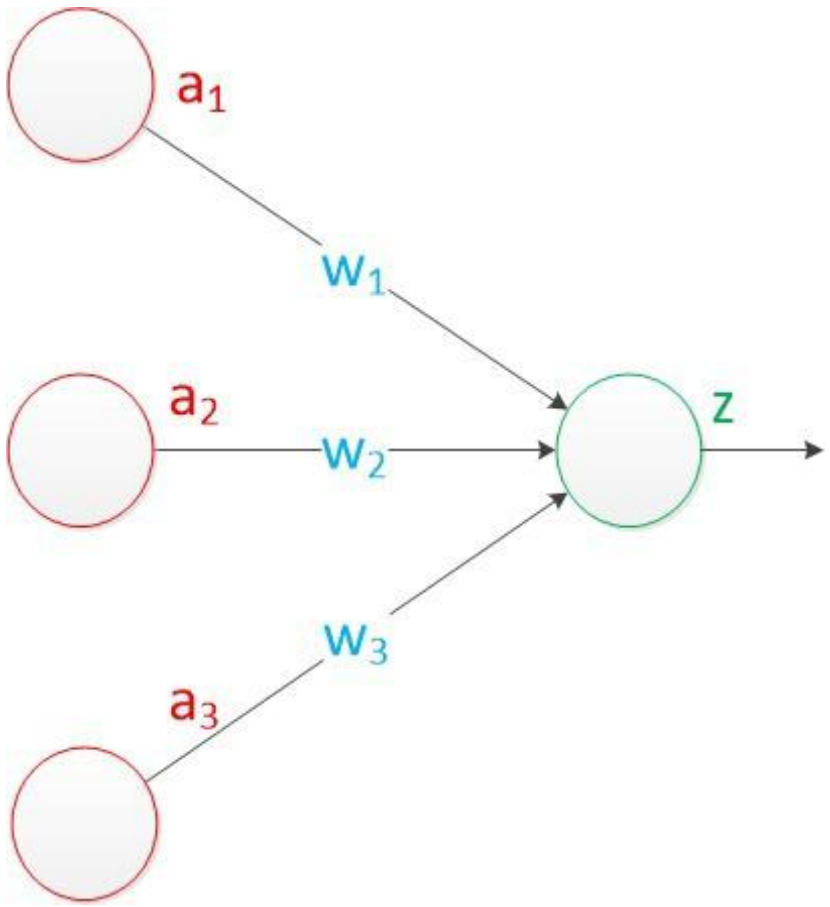
$$z_2 = g(a_1 * w_{2,1} + a_2 * w_{2,2} + a_3 * w_{2,3})$$

Single layer neural network (Perceptron)

- The weights (e.g., wij) in the perceptron are obtained through training
- The perceptron is a logistic regression model that can do linear classification tasks



Single layer neural network (Perceptron)



input

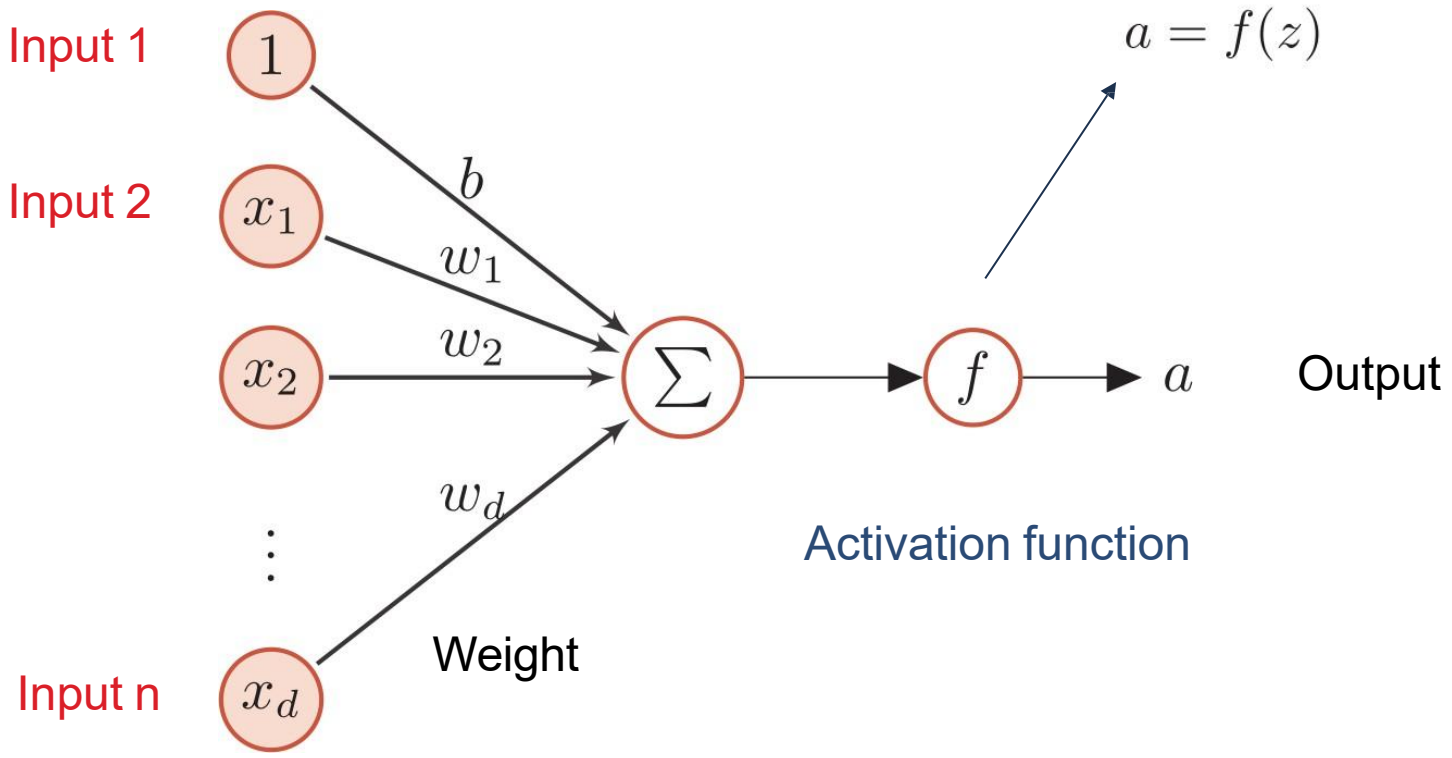
$$z = a^T w$$

Let $a_1 = b$, $w_1 = 1$

$a_k = a_{k+1}$, $w_k = w_{k+1}$

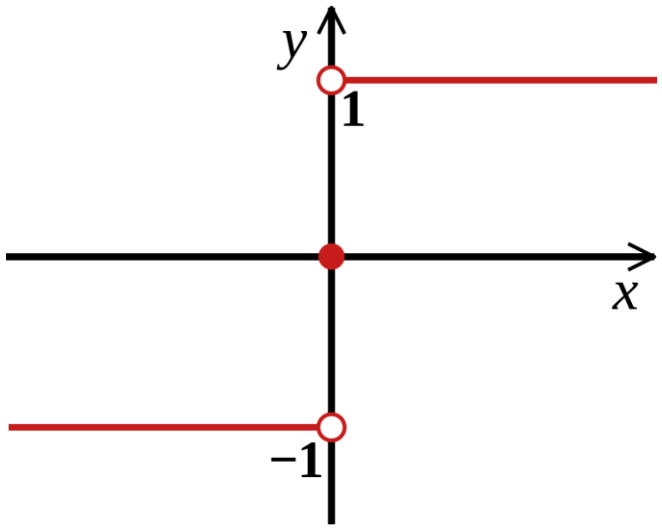
$$z = a^T w + b$$

Single layer neural network (Perceptron)



Sign function

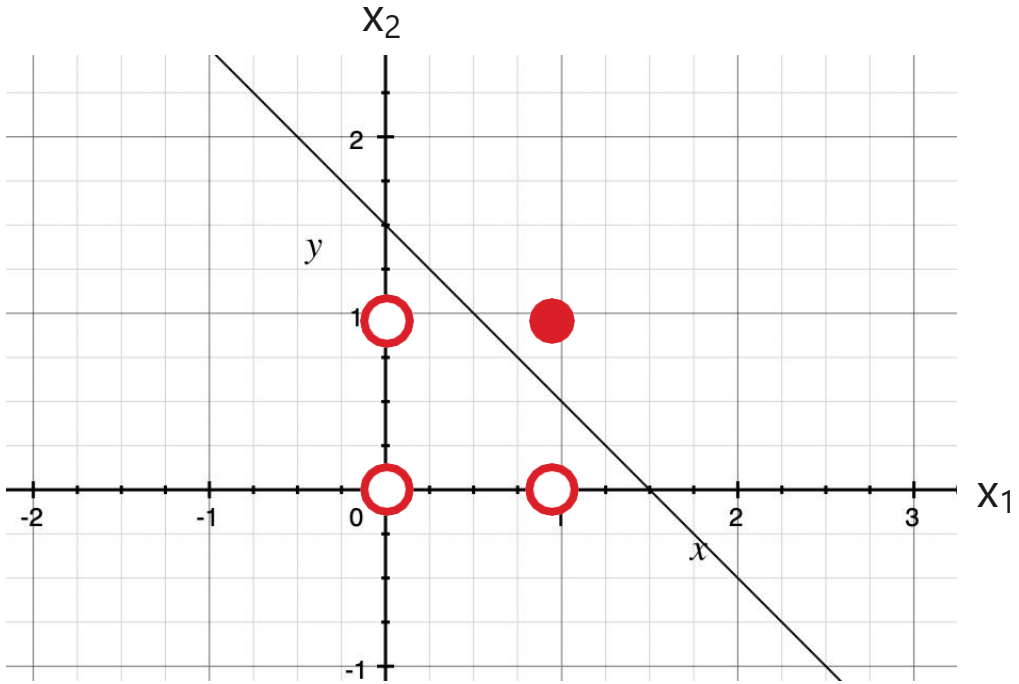
$$f(\text{input}) = \begin{cases} 1, input \geq 0 \\ 0, input < 0 \end{cases}$$



Single layer neural network (Perceptron): AND

- $z = y(x_1, x_2) = f(\omega_1 x_1 + \omega_2 x_2 - \theta)$
- $\omega_1 = \omega_2 = 1$
- $\theta = 1.5$
- $f(\cdot) = \begin{cases} 1, if\ input \geq 0 \\ 0, if\ input < 0 \end{cases}$

x_1	x_2	$x_1 \wedge x_2$
1	1	1
1	0	0
0	1	0
0	0	0
AND GATE		

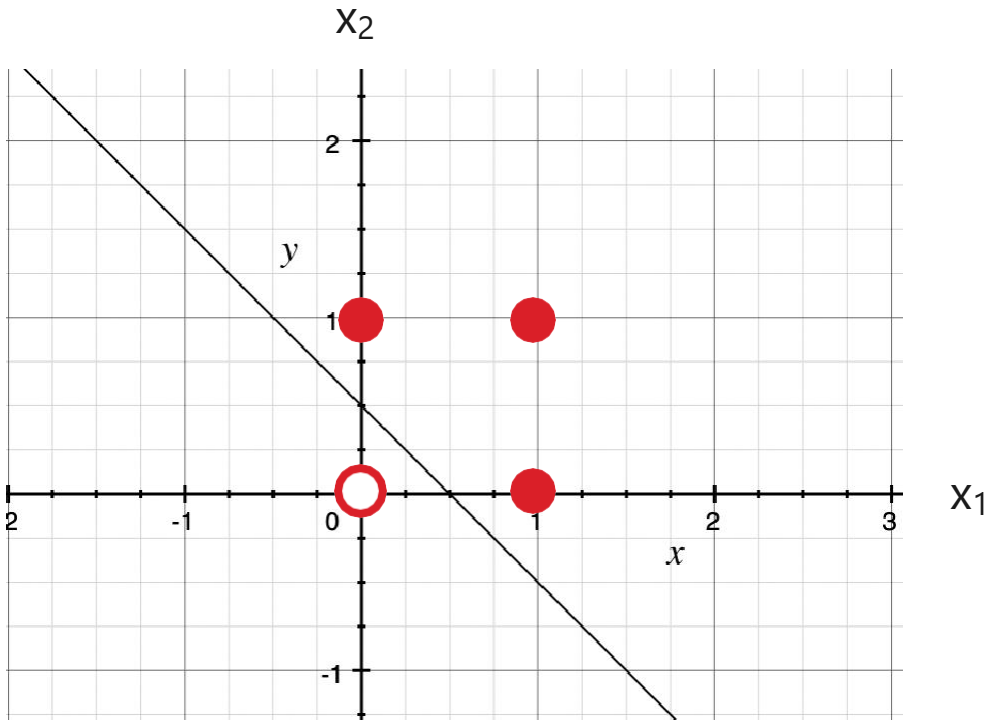


$1^* x_1 + 1^* x_2 - 1.5 = 0$

Single layer neural network (Perceptron): OR

- $z = y(x_1, x_2) = f(\omega_1 x_1 + \omega_2 x_2 - \theta)$
- $\omega_1 = \omega_2 = 1$
- $\theta = 0.5$
- $f(\cdot) = \begin{cases} 1, if\ input \geq 0 \\ 0, if\ input < 0 \end{cases}$

x_1	x_2	$x_1 \vee x_2$
1	1	1
1	0	1
0	1	1
0	0	0
OR GATE		

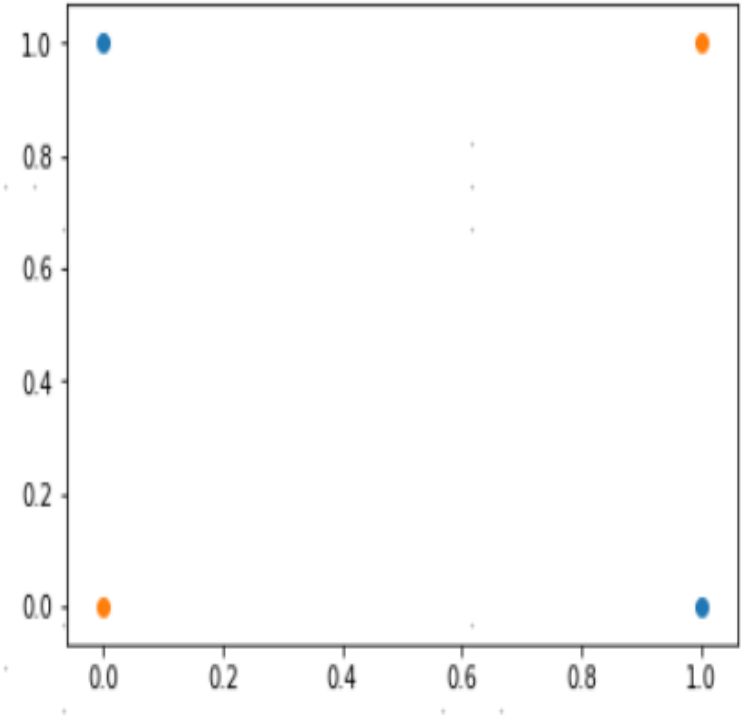


$1 * x_1 + 1 * x_2 - 0.5 = 0$

Single layer neural network (Perceptron): OR

- $z = y(x_1, x_2) = f(\omega_1 x_1 + \omega_2 x_2 - \theta)$
- $\omega_1 = \omega_2 = 1$
- $\theta = 0.5$
- $f(\cdot) = \begin{cases} 1, if\ input \geq 0 \\ 0, if\ input < 0 \end{cases}$

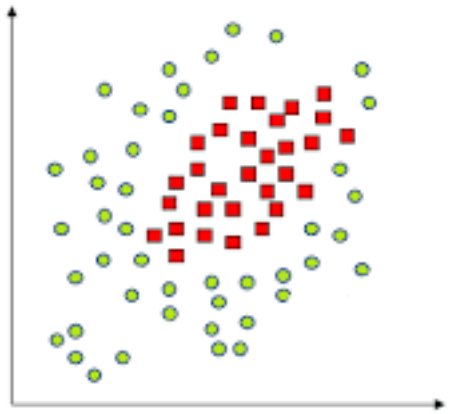
(x1, x2)	y
(0, 0)	0
(0, 1)	1
(1, 0)	1
(1, 1)	0



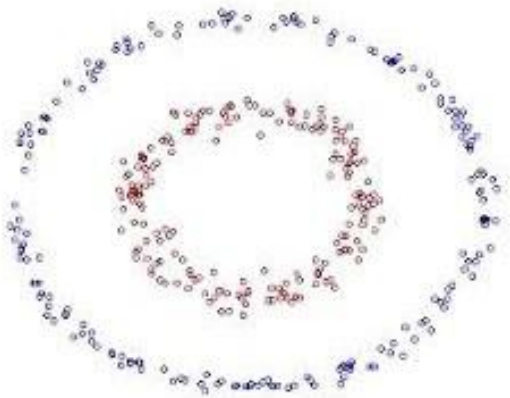
Linear inseparable

Linear inseparable

- We cannot use a straight line to divide the two types of data

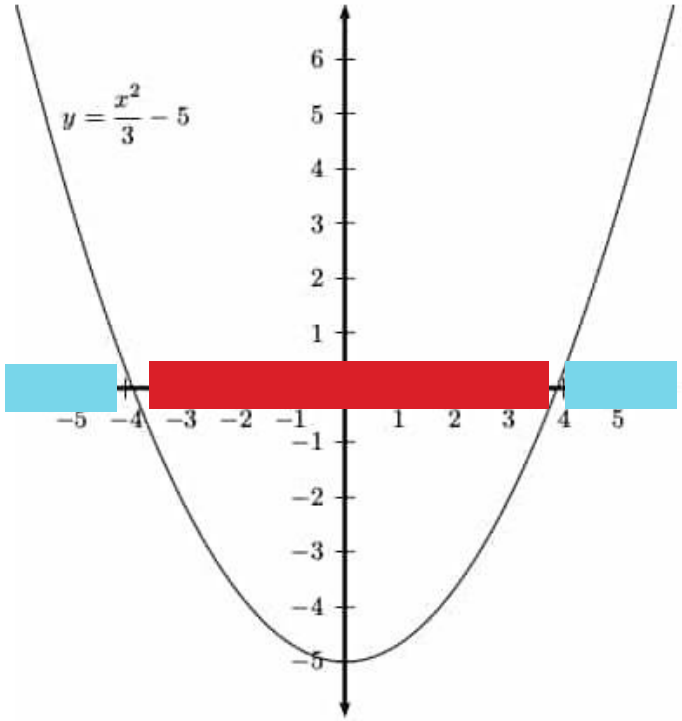


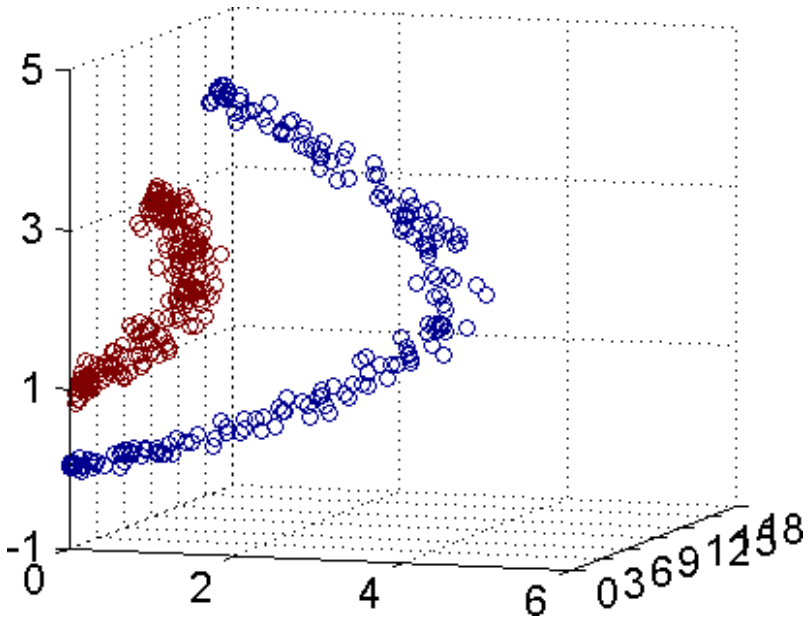
- Such data sets are very common in practical applications, such as: face images, text documents



Linear inseparable

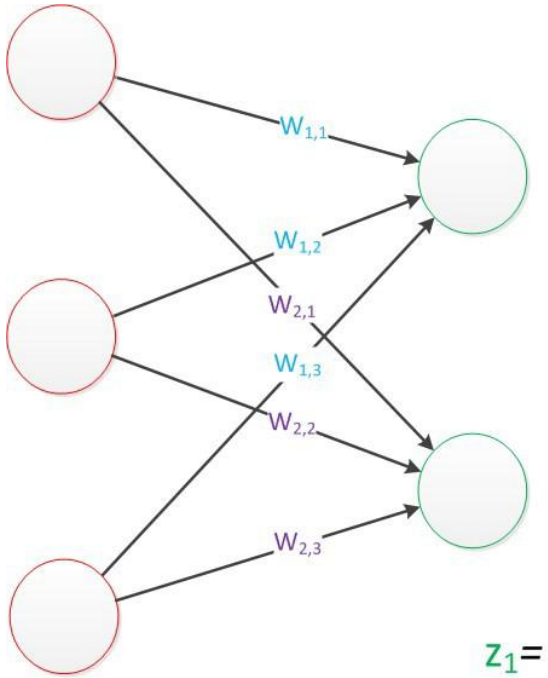
- The solution to linear inseparability is to use the kernel function mapping to high-dimensional space





Two-layer Neural Network

Two-layer neural network

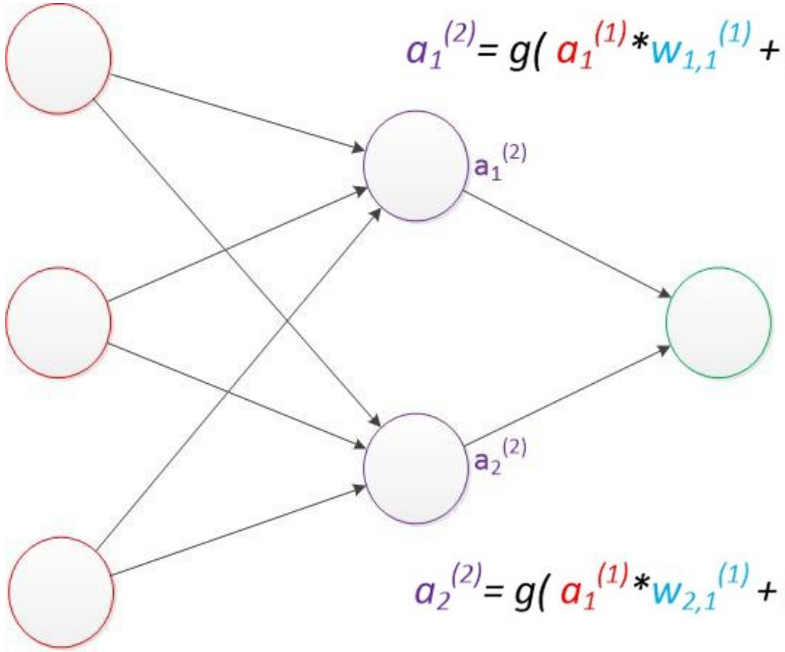


input

$$z_1 = g(\, a_1 * w_{1,1} + a_2 * w_{1,2} + a_3 * w_{1,3} \,)$$

$$z_2 = g(\, a_1 * w_{2,1} + a_2 * w_{2,2} + a_3 * w_{2,3} \,)$$

$$a_1^{(2)} = g(\, a_1^{(1)} * w_{1,1}^{(1)} + a_2^{(1)} * w_{1,2}^{(1)} + a_3^{(1)} * w_{1,3}^{(1)} \,)$$

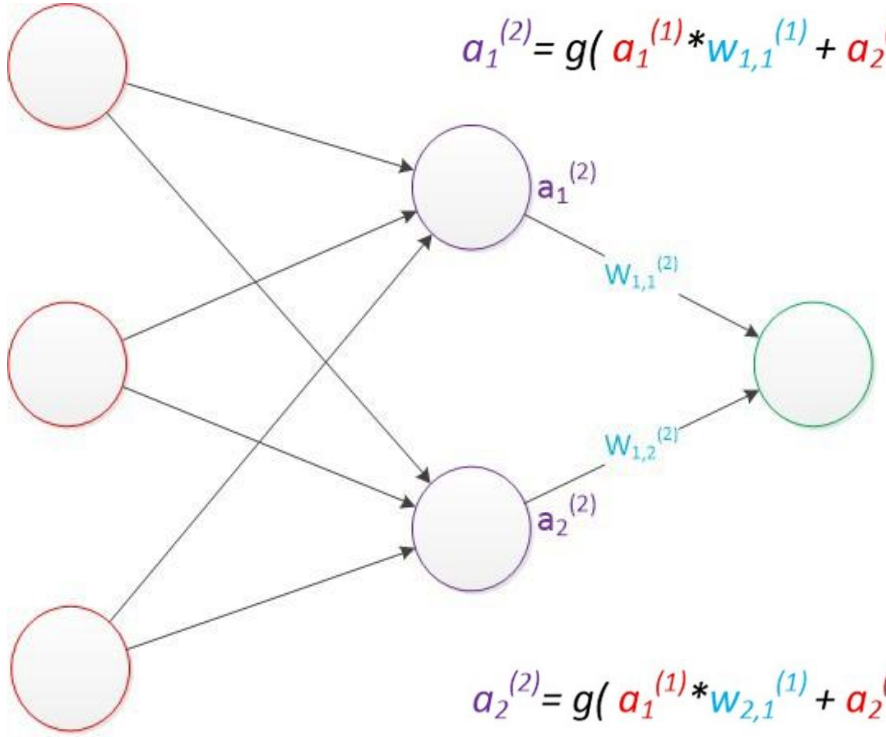


input

$$a_2^{(2)} = g(\, a_1^{(1)} * w_{2,1}^{(1)} + a_2^{(1)} * w_{2,2}^{(1)} + a_3^{(1)} * w_{2,3}^{(1)} \,)$$

Two-layer neural network

$$a_1^{(2)} = g(\, a_1^{(1)} * w_{1,1}^{(1)} + a_2^{(1)} * w_{1,2}^{(1)} + a_3^{(1)} * w_{1,3}^{(1)} \,)$$

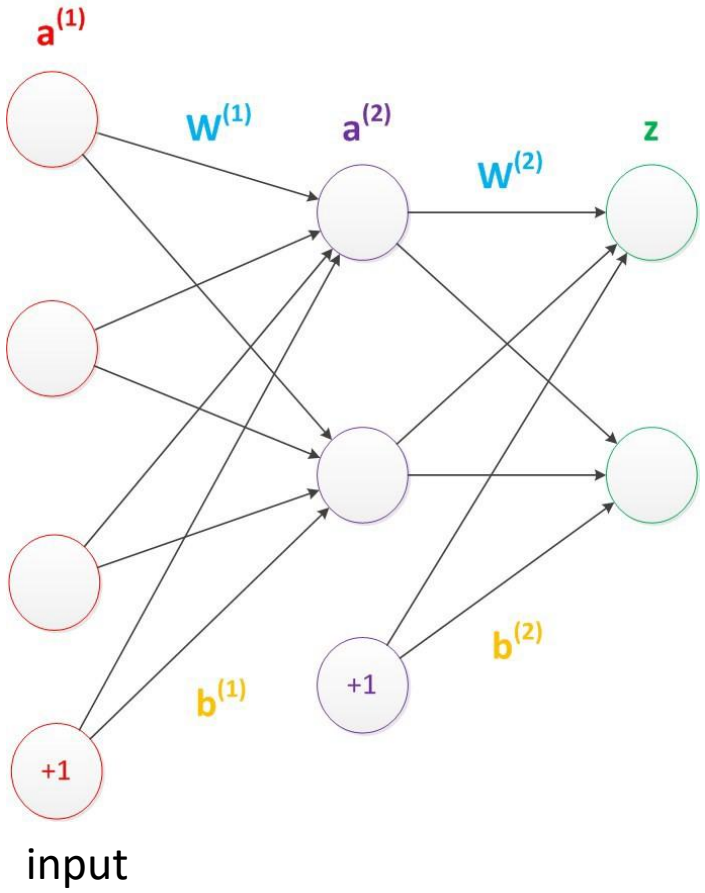


$$z = g(\, a_1^{(2)} * W_{1,1}^{(2)} + a_2^{(2)} * W_{1,2}^{(2)} \,)$$

$$a_2^{(2)} = g(\, a_1^{(1)} * w_{2,1}^{(1)} + a_2^{(1)} * w_{2,2}^{(1)} + a_3^{(1)} * w_{2,3}^{(1)} \,)$$

input

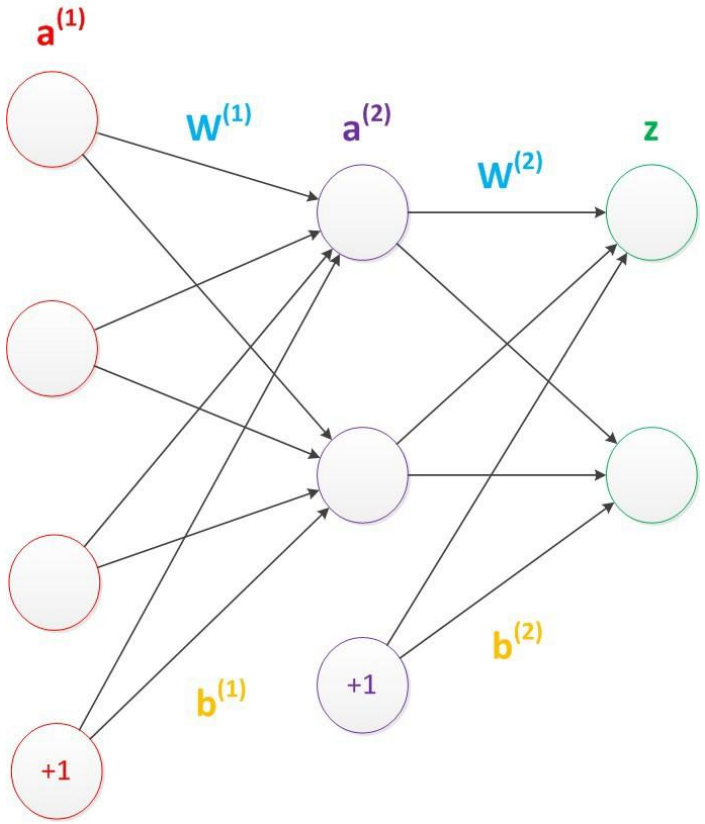
Two-layer neural network (including the bias node)



$$g(\mathbf{W}^{(1)} * \mathbf{a}^{(1)} + \mathbf{b}^{(1)}) = \mathbf{a}^{(2)}$$

$$g(\mathbf{W}^{(2)} * \mathbf{a}^{(2)} + \mathbf{b}^{(2)}) = \mathbf{z}$$

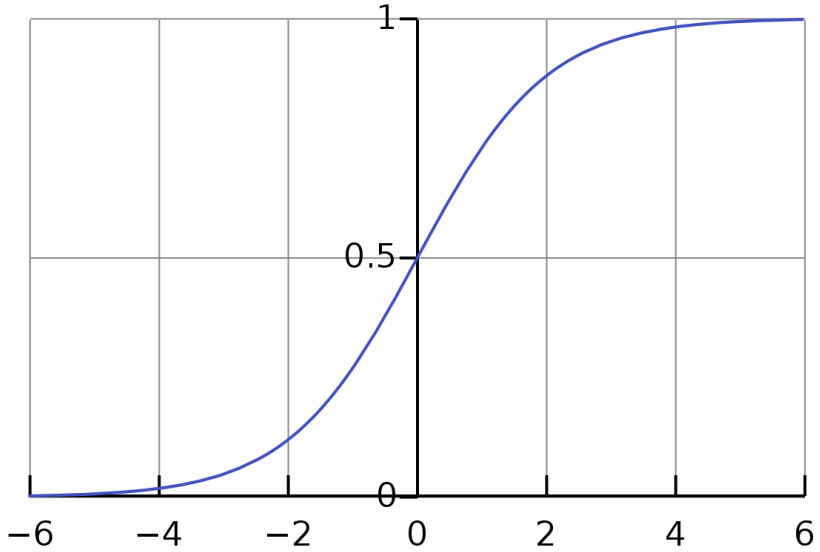
Two-layer neural network (including the bias node)



$$g(\mathbf{W}^{(1)} * \mathbf{a}^{(1)} + \mathbf{b}^{(1)}) = \mathbf{a}^{(2)}$$

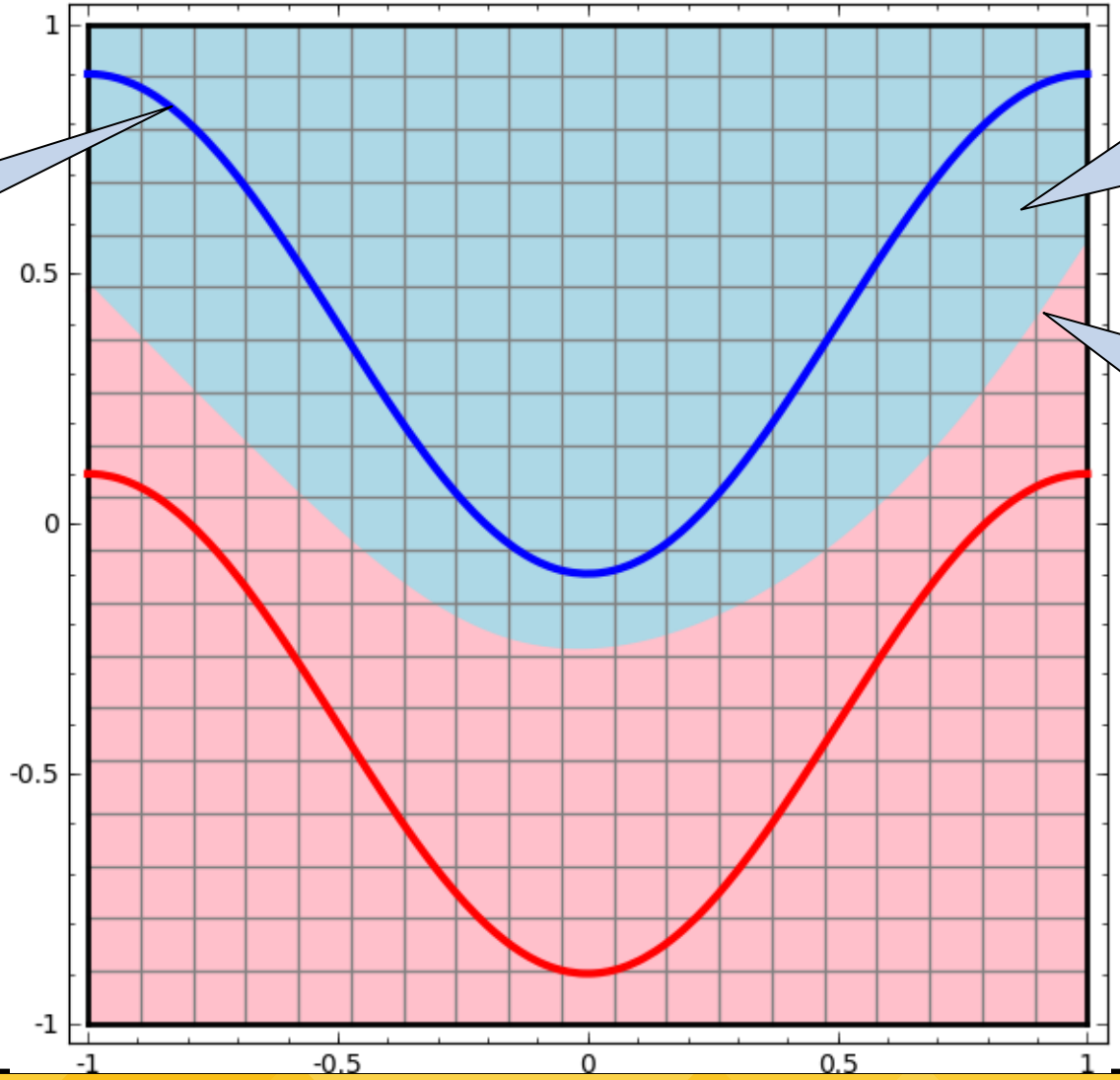
$$g(\mathbf{W}^{(2)} * \mathbf{a}^{(2)} + \mathbf{b}^{(2)}) = \mathbf{z}$$

We used sigmoid function for g()



Non-linear classification task

The red and blue lines represent data

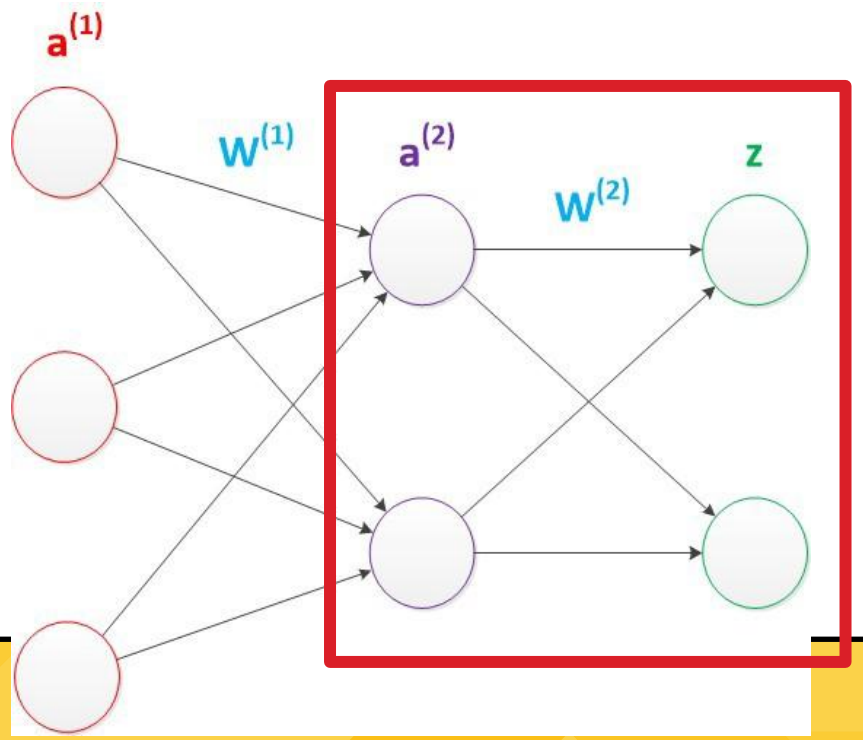


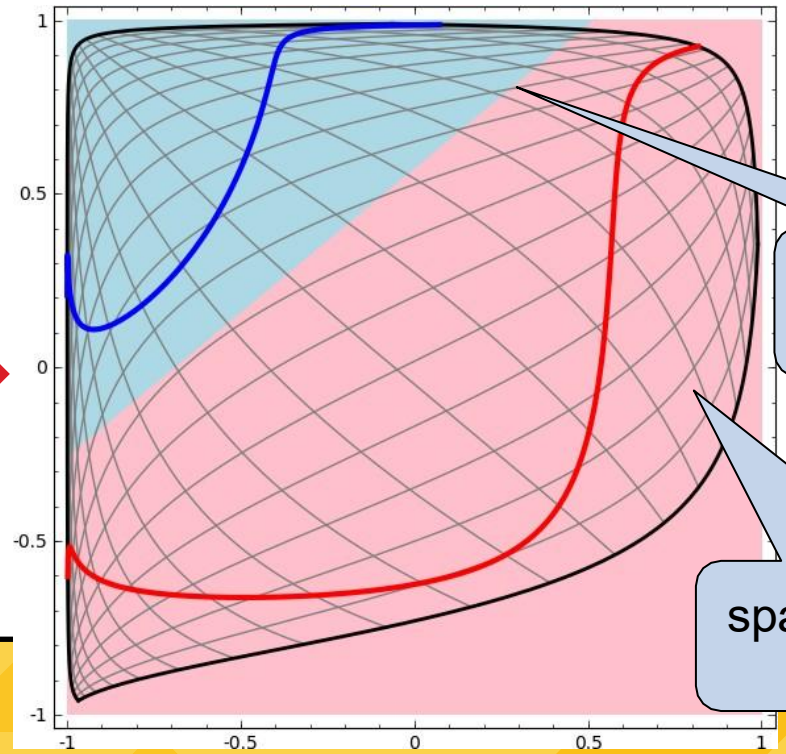
The red area and blue area represent the area divided by the neural network

The dividing line is generated by the NN. The curve is very smooth by NN.

Non-linear classification task

- Single-layer networks can only do linear classification tasks
  - Because its decision boundary is a hyperplane
- Why can a combination of two linear classification tasks be used for nonlinear classification tasks?



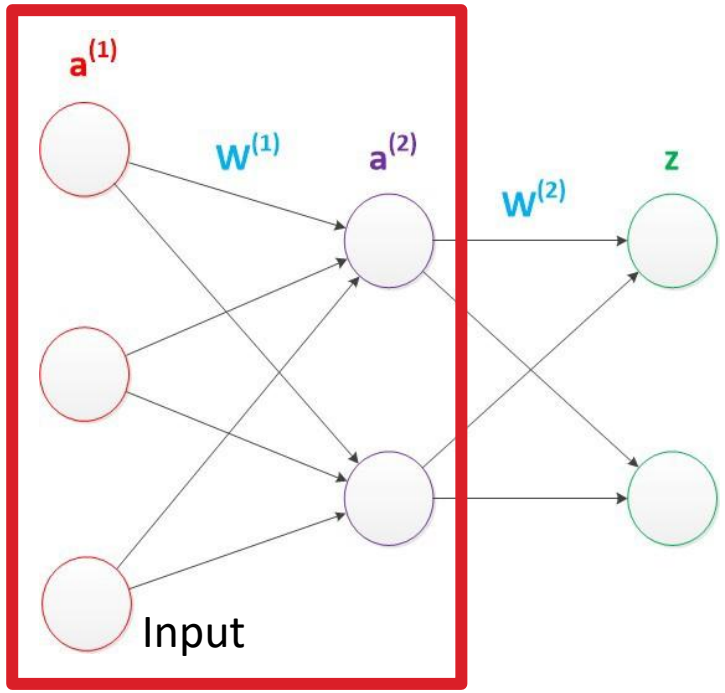


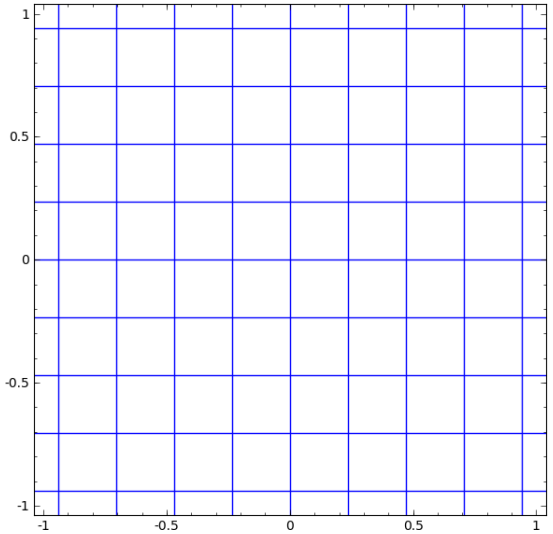
The dividing line is straight.

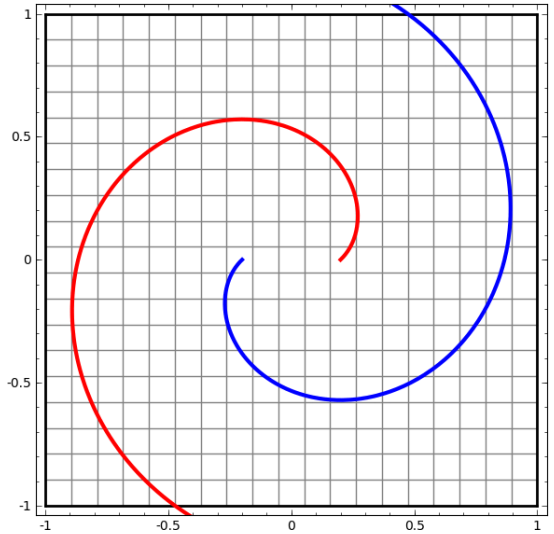
spatial transformation of the data

Non-linear classification task

- In the two-layer neural network, the hidden layer performs a spatial transformation on the original data so that it can be linearly classified

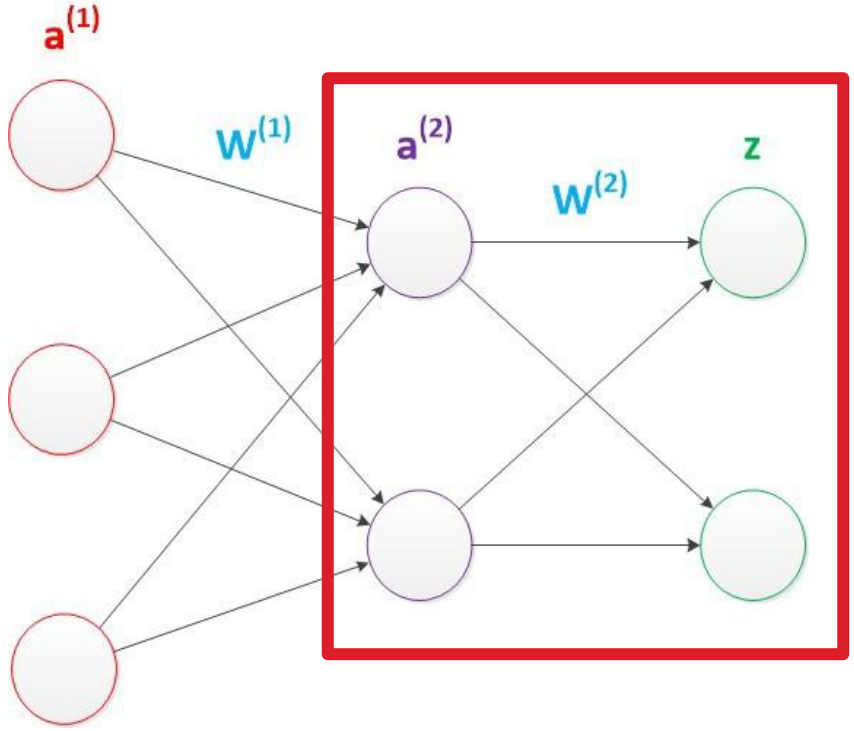


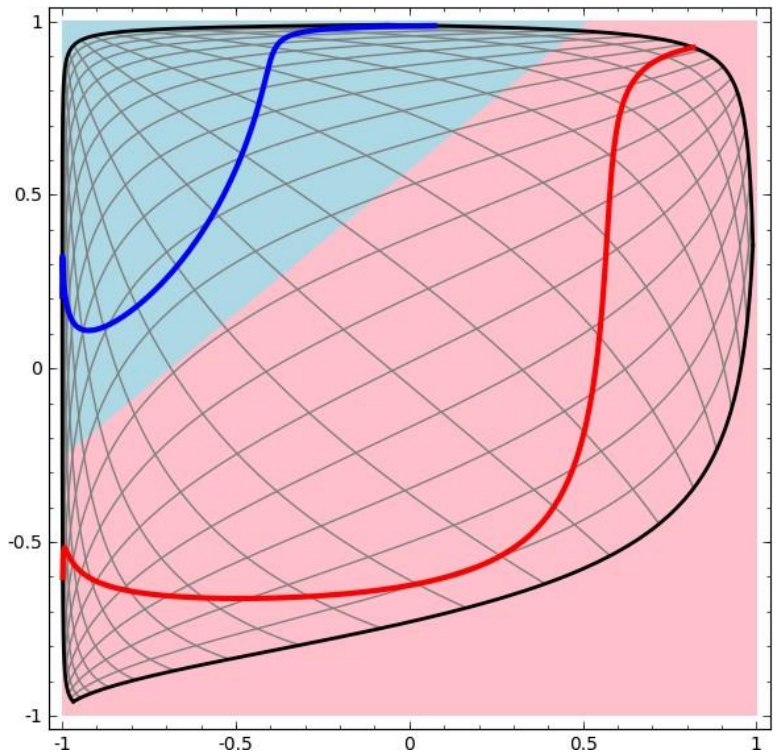




Non-linear classification task

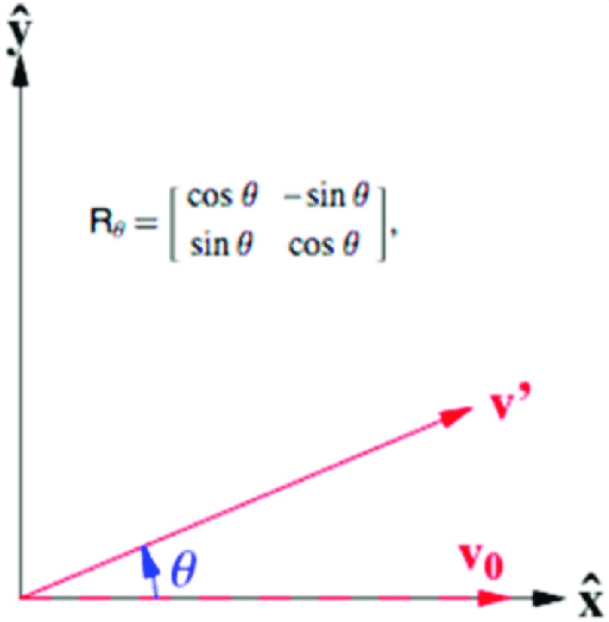
- The decision boundary of the output layer draws a linear classification boundary and classifies it

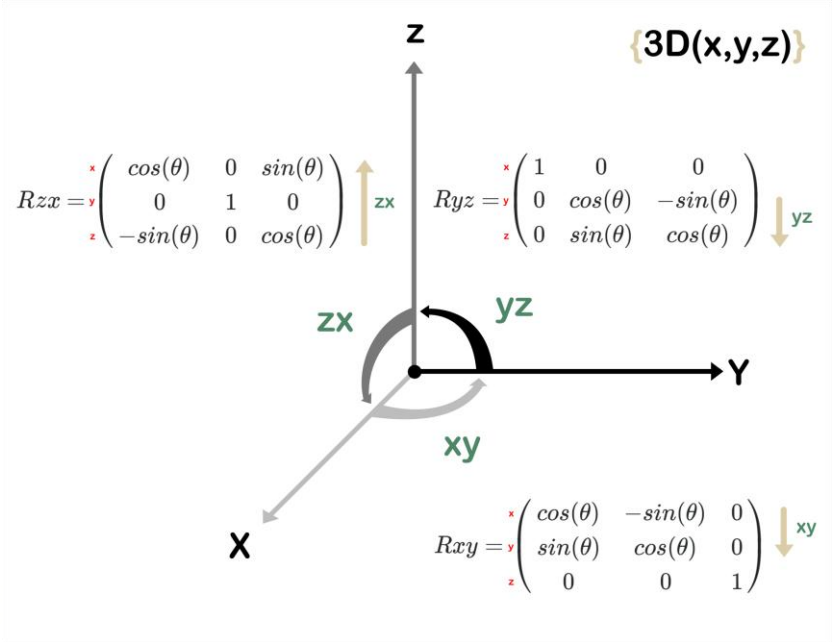


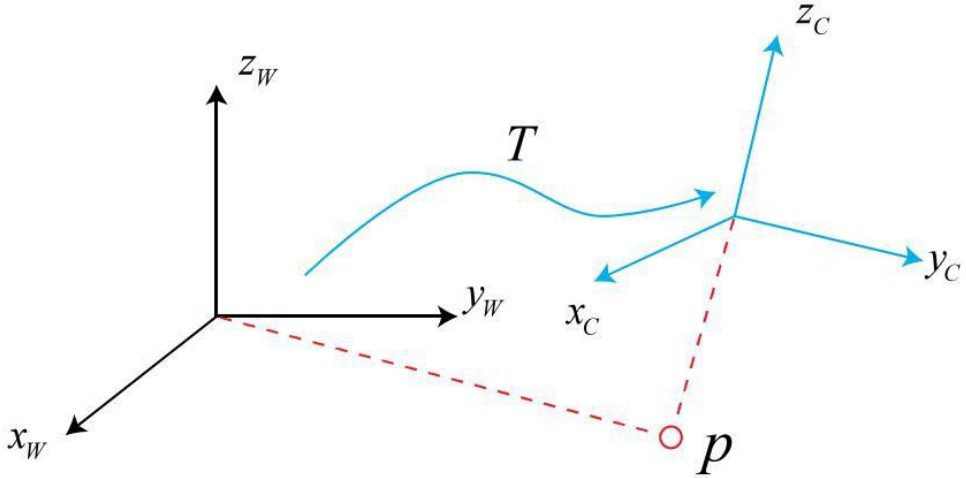


Non-linear classification task

- Multiplying a matrix and a vector is essentially a transformation of the coordinate space of the vector

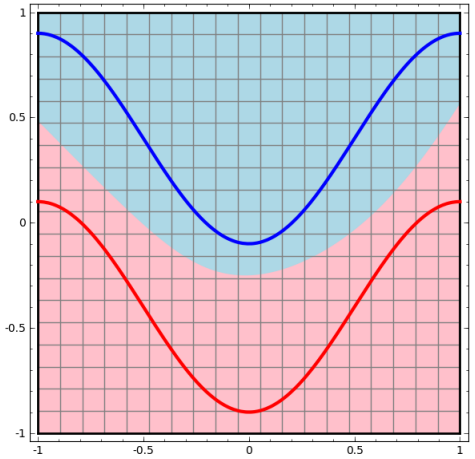


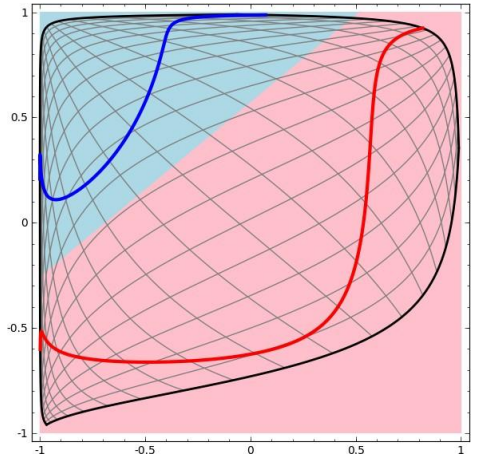


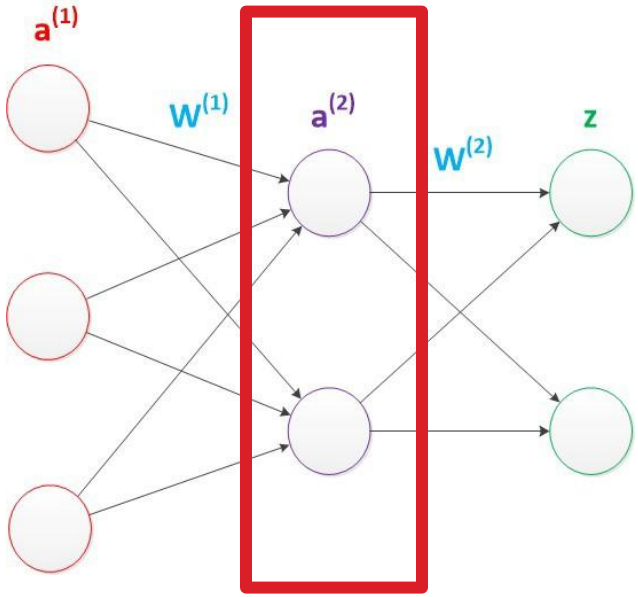


Non-linear classification task

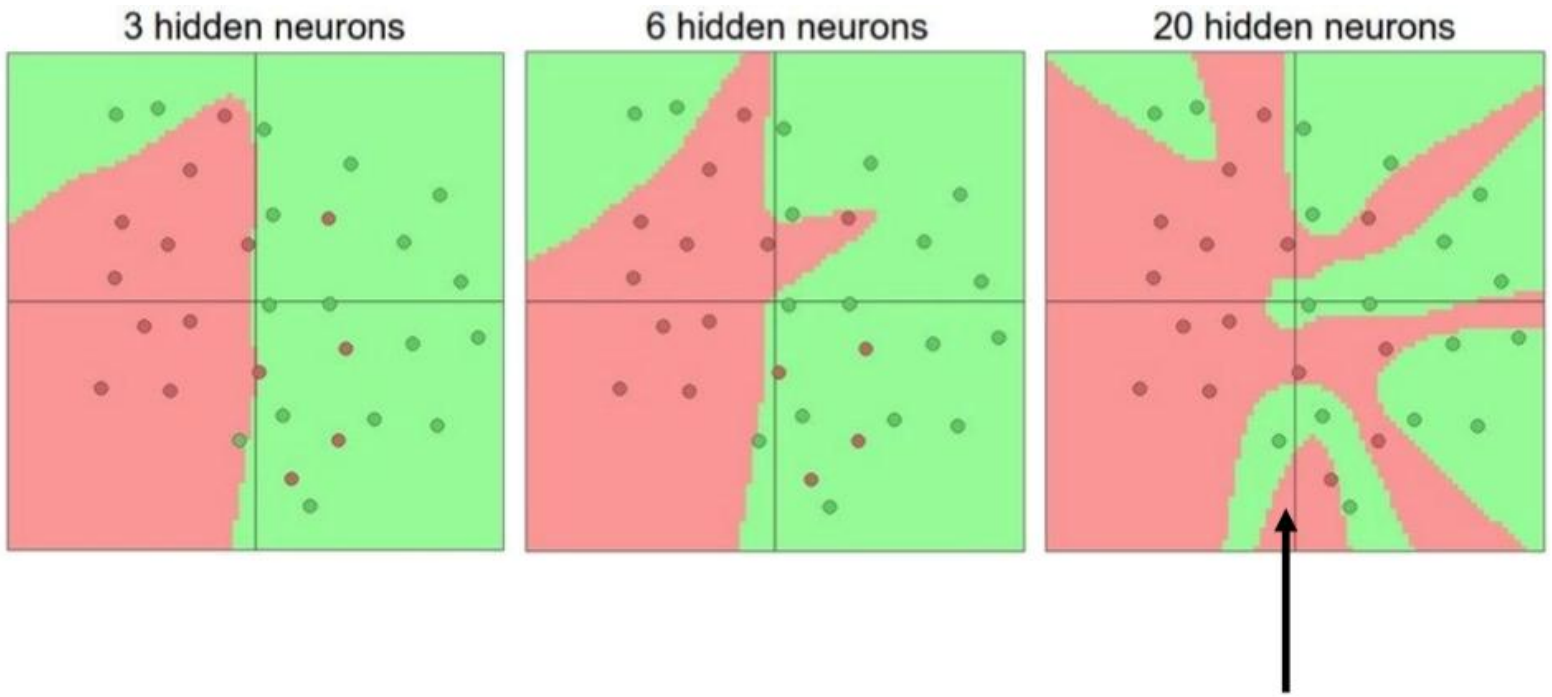
- The function of the parameter matrix of the hidden layer is to make the original coordinate space of the data from linear indivisible to linear separable





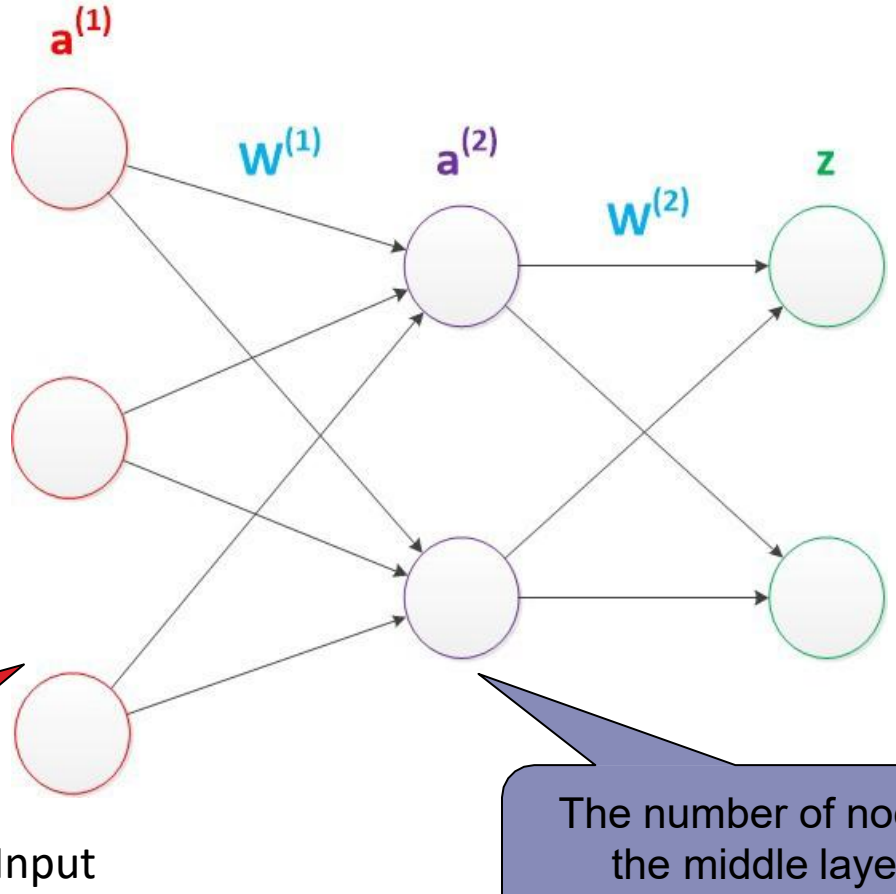


Number of Layers and Number of Neurons



more neurons/layers = more capacity

Design of Two-layer neural network



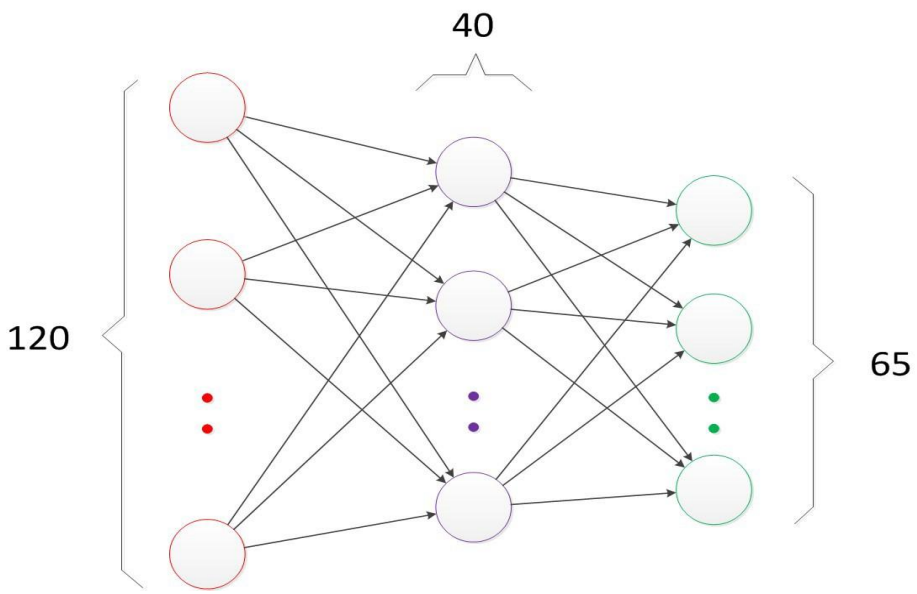
The number of nodes in the output layer must match the dimensionality of the target

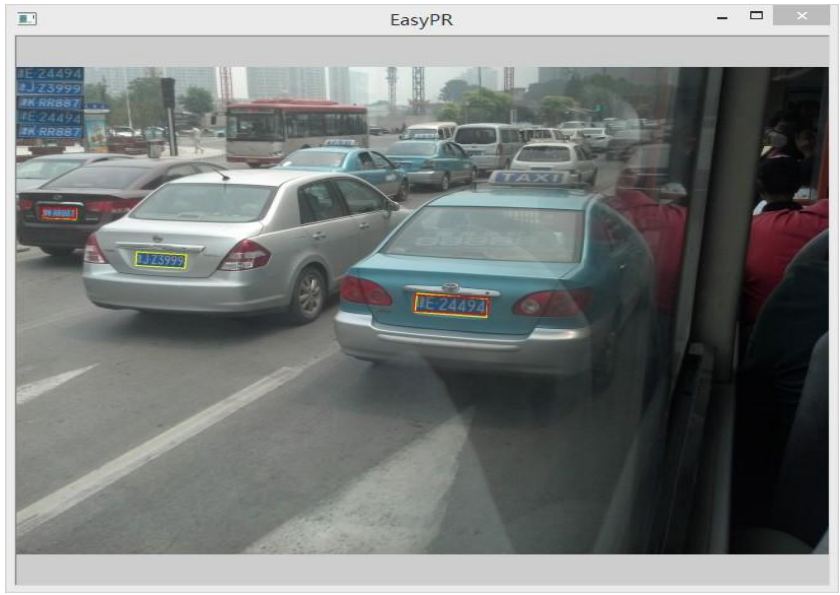
The number of nodes in the input layer needs to match the dimension of the feature

The number of nodes in the middle layer is specified by the designer

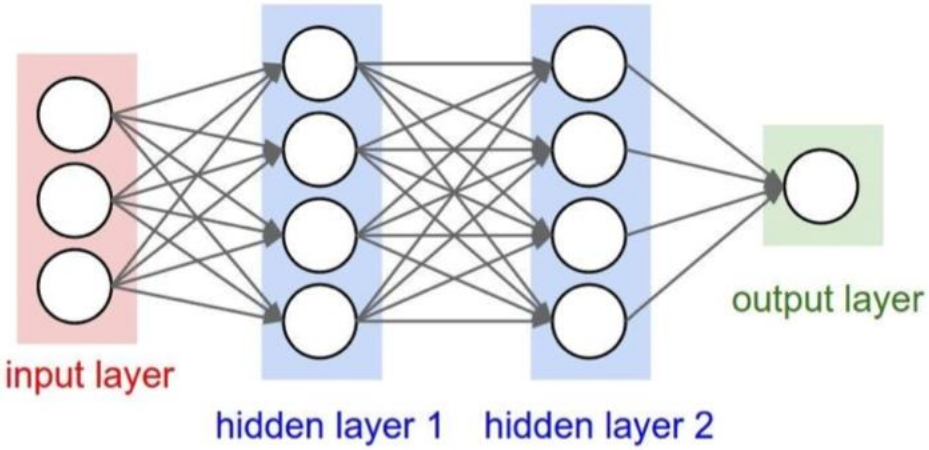
Design of Two-layer neural network

- Plate recognition (EasyPR): the input is a 120-dimensional vector. The output is 65 types of text. They found the performance is the best when hidden layer number is 40





Implementation of feed-forward computation of a neural network

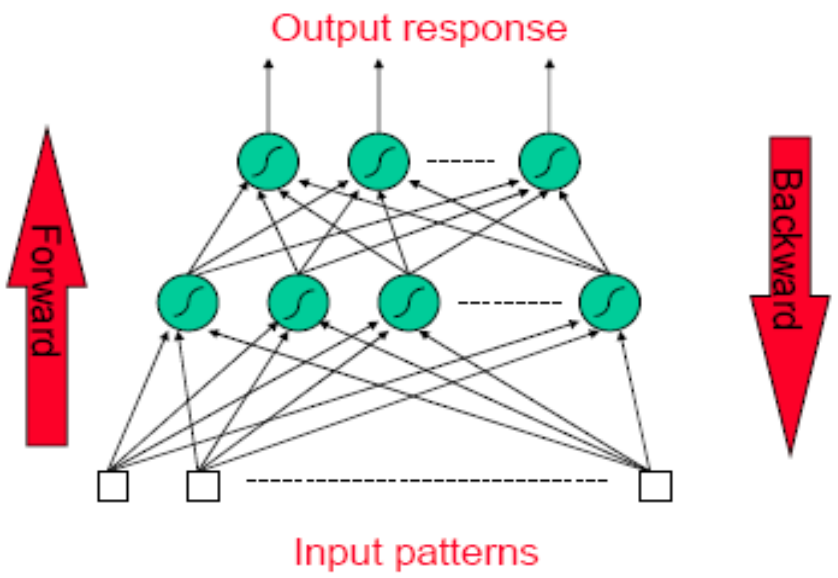



```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Training & Back Propagation

Back Propagation

- Suppose
  - We have built a neural network with two layers
  - We have built a classifier (e.g. Softmax)
  - We have defined loss function (e.g. binary cross entropy)
- Let's do gradient descent by back propagation!
  - Forward pass phase: computes 'functional signal', feed forward propagation of input through network
  - Backward pass phase: computes 'error signal', propagates the error backwards through network starting at output units (where the error is the difference between actual and desired output values)
- How to calculate gradients?

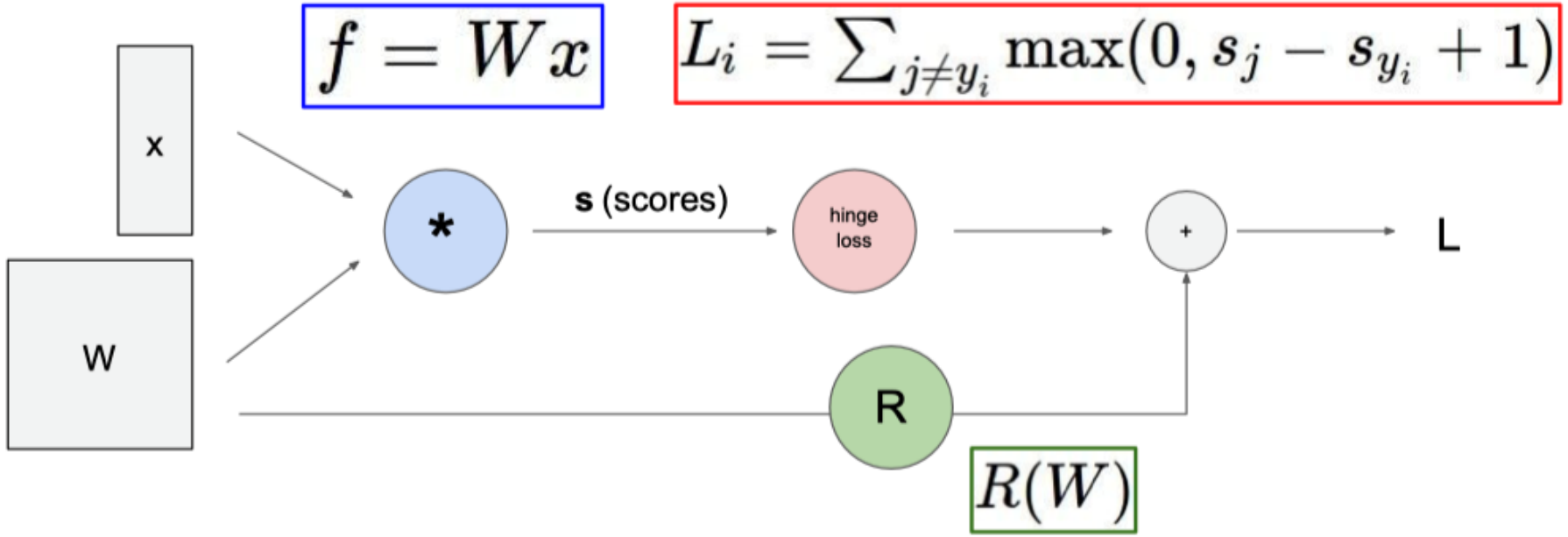


(Bad) Idea: Derive gradient on paper

- Problem 1: Very tedious: Lots of matrix calculus, need lots of paper
- Problem 2: What if we want to change loss?
  - E.g. use logistic loss instead of hinge loss? Need to re-derive from scratch
- Problem 3: Not feasible for very complex models!
  - E.g. A neural network with 100 layers

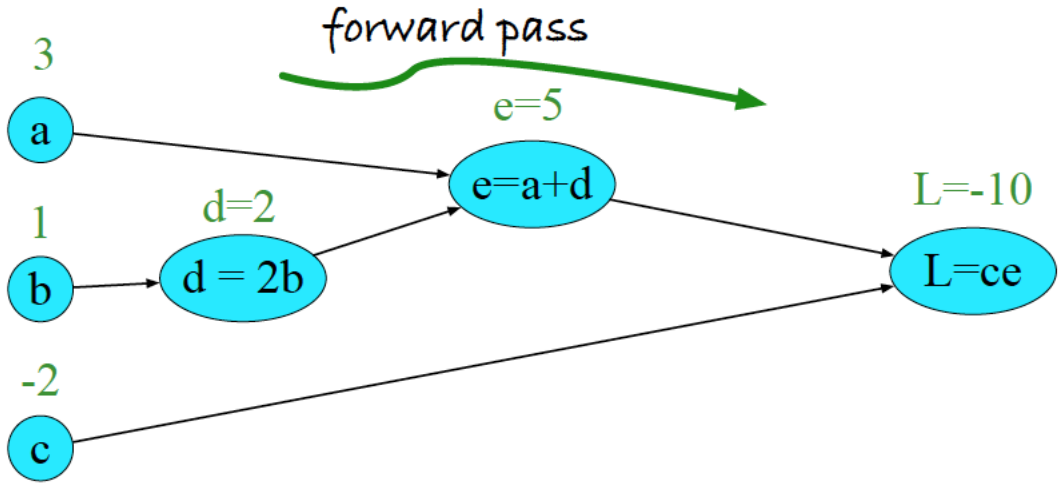
Better Idea: Computational graphs + Backpropagation

- $L = \sum_i^N L_i + R(W)$
- In practice:
  - Computational graphs
  - Backpropagation by auto differentiation



Backpropagation Intuition

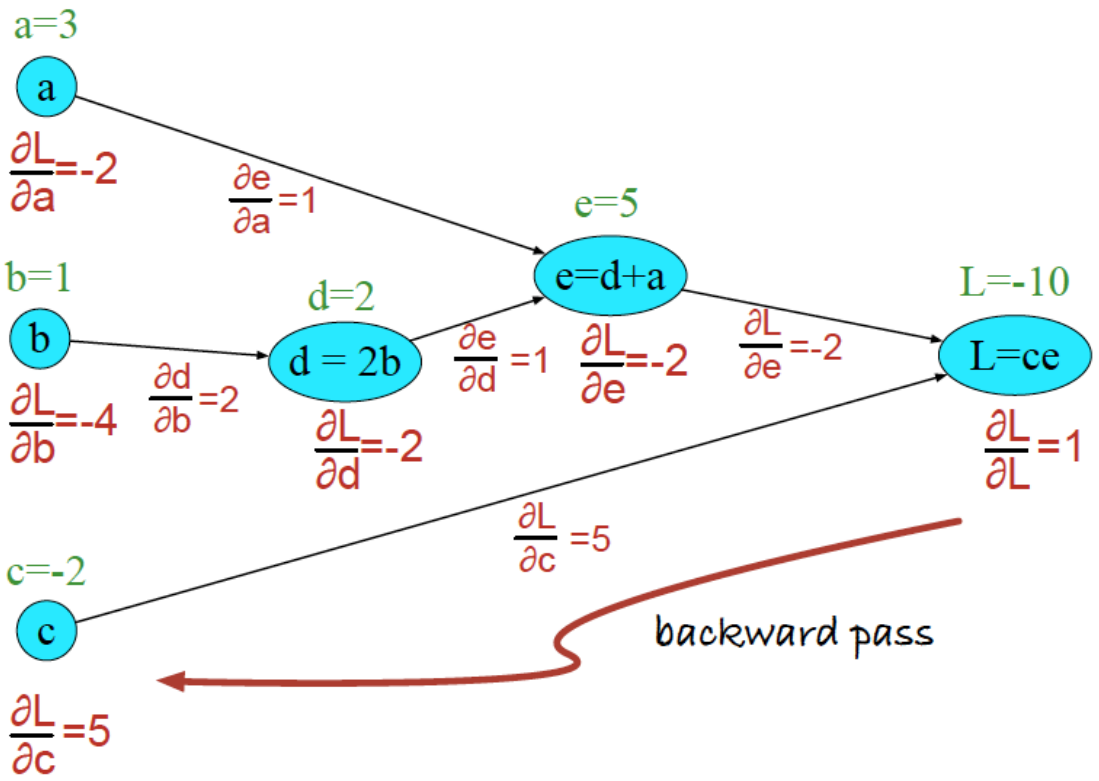
- Given equations:
  - $d = 2 * b$
  - $e = a + d$
  - $L = c * e$
- Computation graph



- Chain rule (partial derivative)
  - $f(x) = u(w(x))$
  - $\frac{\partial f}{\partial x} = \frac{\partial v}{\partial w}\frac{\partial w}{\partial x}$

Backpropagation Intuition

- Given equations:
  - $d = 2 * b$
  - $e = a + d$
  - $L = c * e$
- Loss function
  - $L(a, b, c) = c(a + 2b) = ce$
- Derivative
  - $\frac{\partial L}{\partial c} = e$
  - $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} = c \cdot 1 = c$
  - $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b} = c \cdot 1 \cdot 2 = 2c$

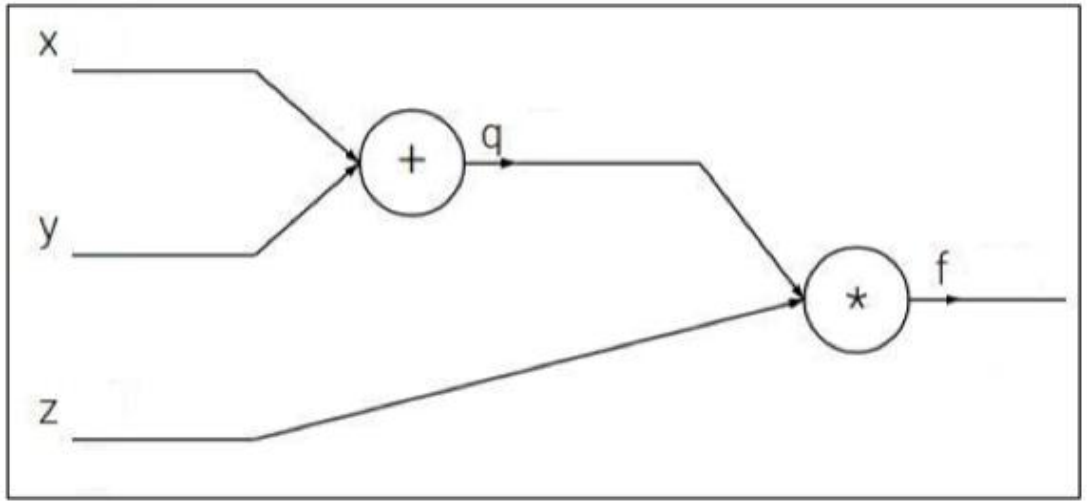


Example - Backpropagation Exercise

- Given equation:
  - $q = x + y$
  - $f = q * z$
  - Given $x = -2, y = 5, z = -4$
- Show backpropagation circuit diagram with corresponding derivative values.

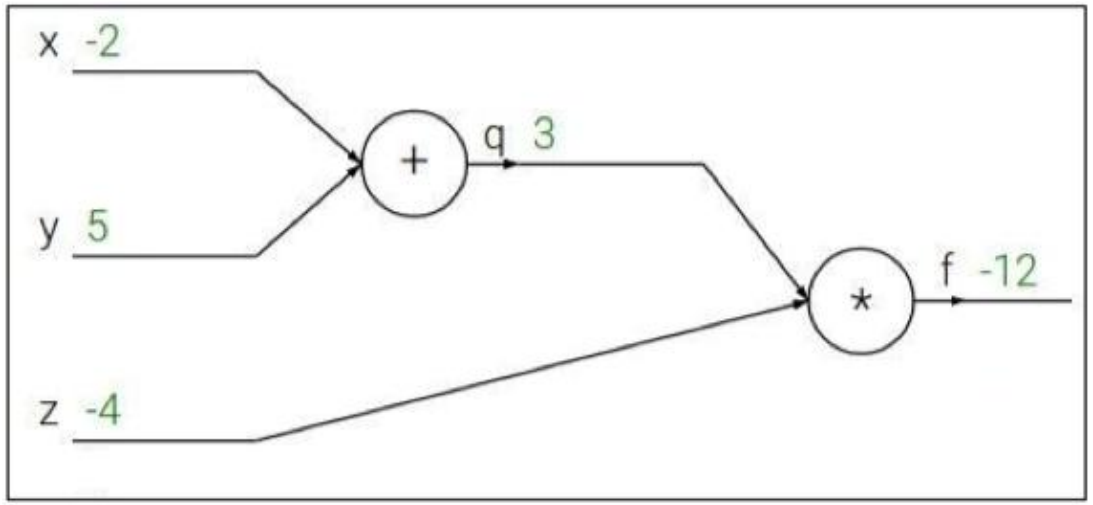
Example - Answer

- Given equation:
  - $q = x + y$
  - $f = q * z$
  - Given $x = -2, y = 5, z = -4$
  - $f(x, y, z) = (x + y)z$



Example - Answer

- Given equation:
  - $q = x + y$
  - $f = q * z$
  - Given $x = -2, y = 5, z = -4$
  - $f(x, y, z) = (x + y)z$

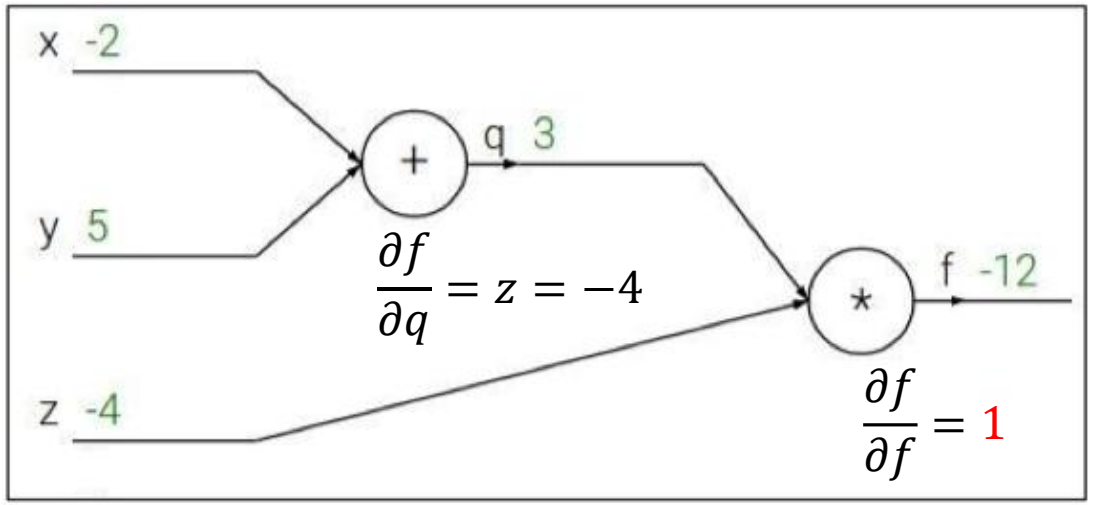


Example - Answer

- Given equation:
  - $q = x + y$
  - $f = q * z$
  - Given $x = -2, y = 5, z = -4$
  - $f(x, y, z) = (x + y)z$

$$\frac{\partial f}{\partial x} = z = -4$$

$$\frac{\partial f}{\partial y} = z = -4$$

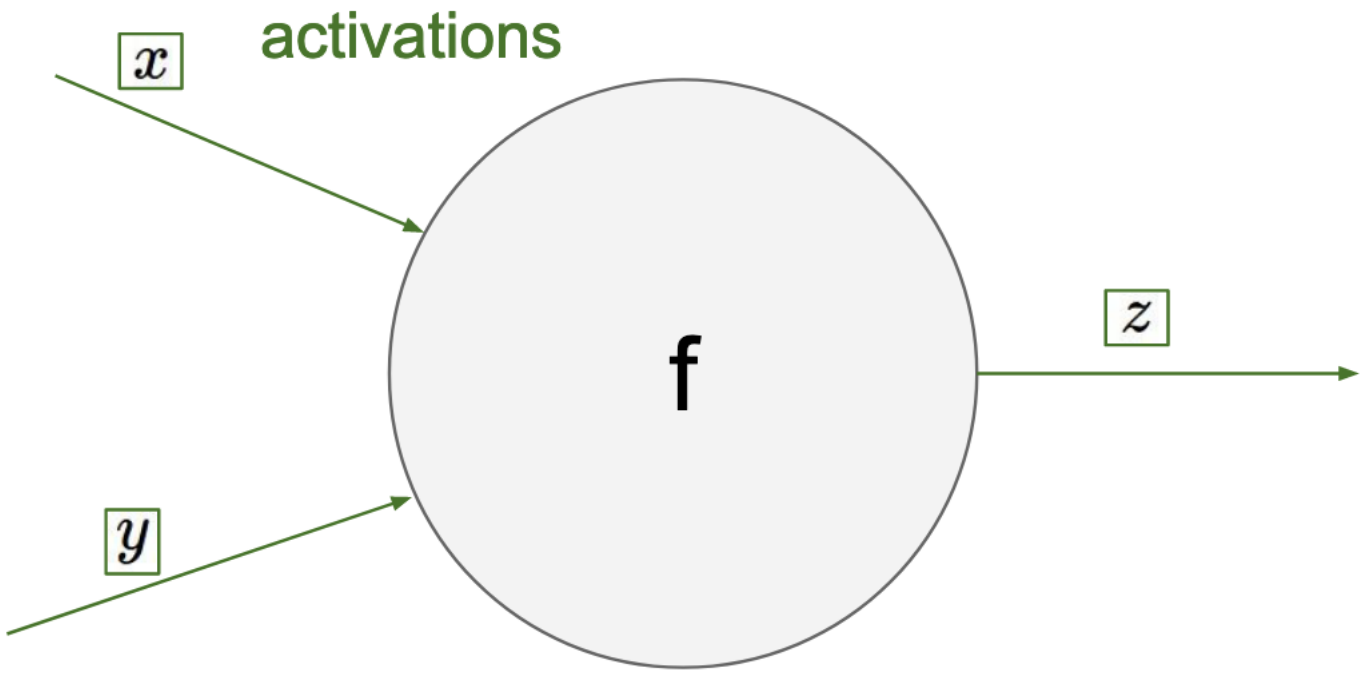


$$\frac{\partial f}{\partial q} = z = -4$$

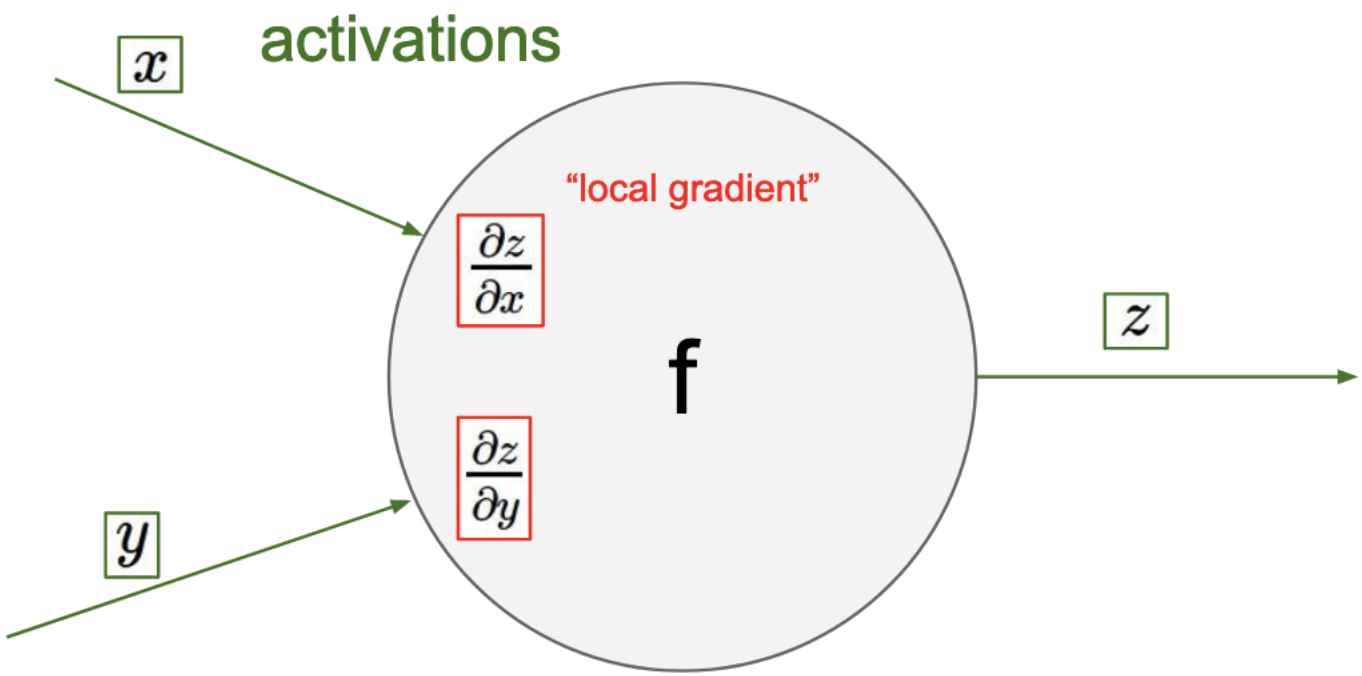
$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial z} = q = 3$$

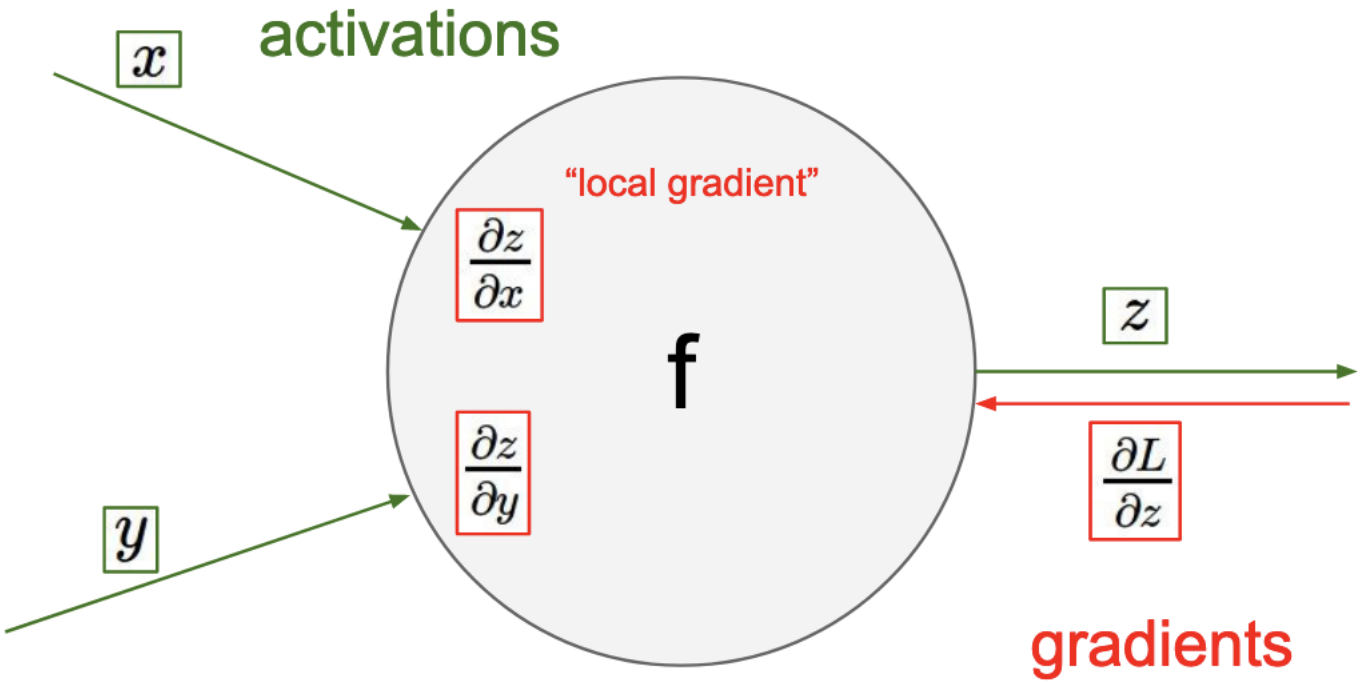
BP local gradient



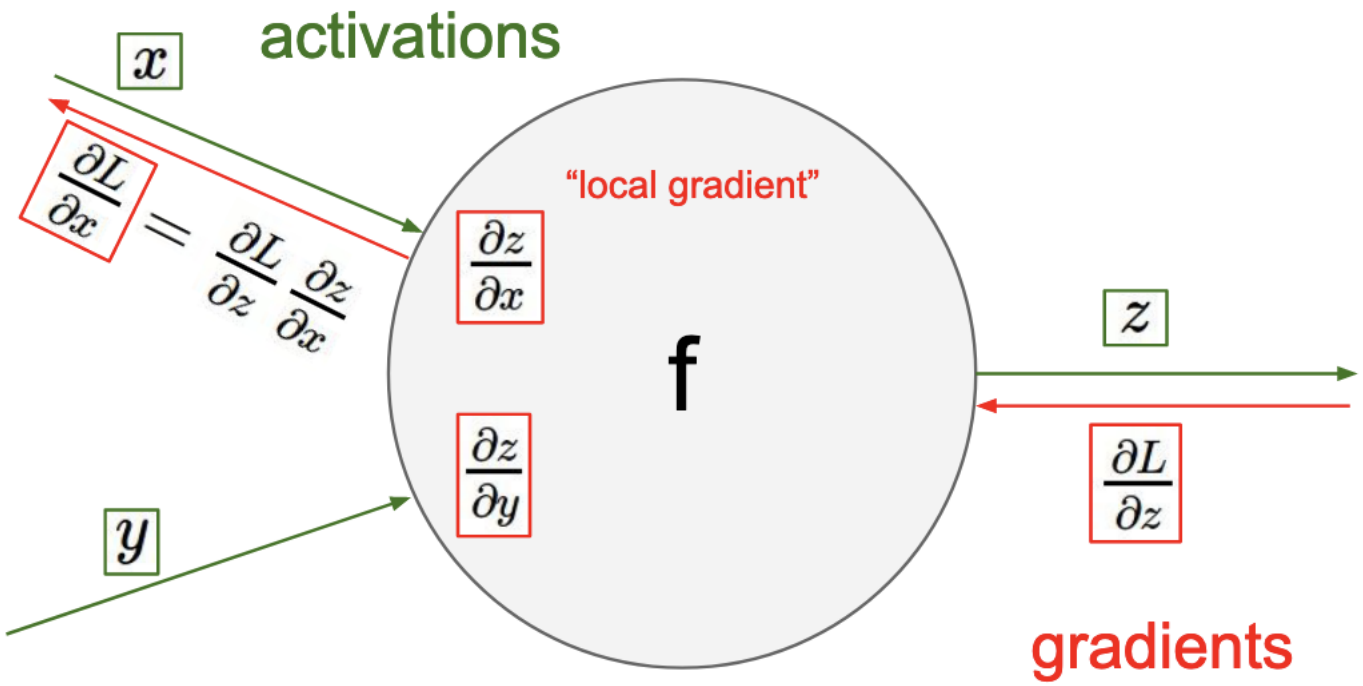
BP local gradient



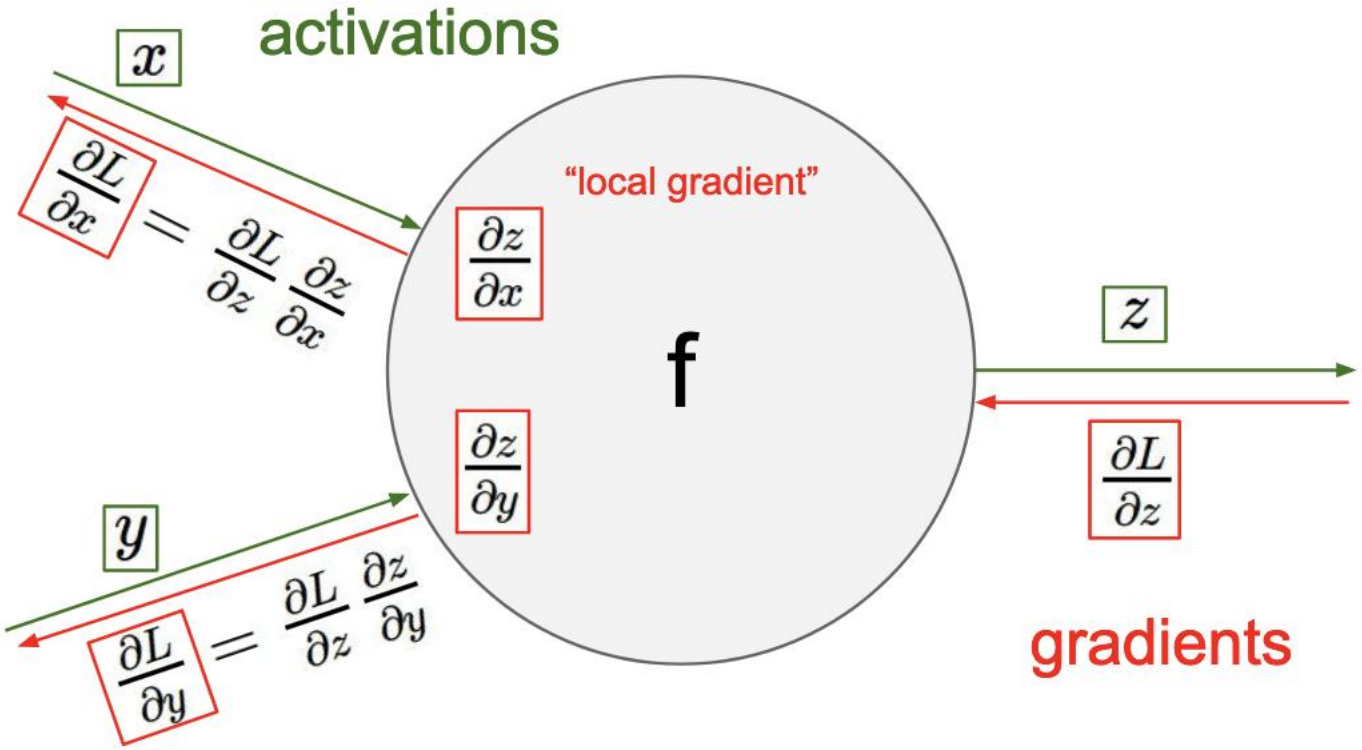
BP local gradient



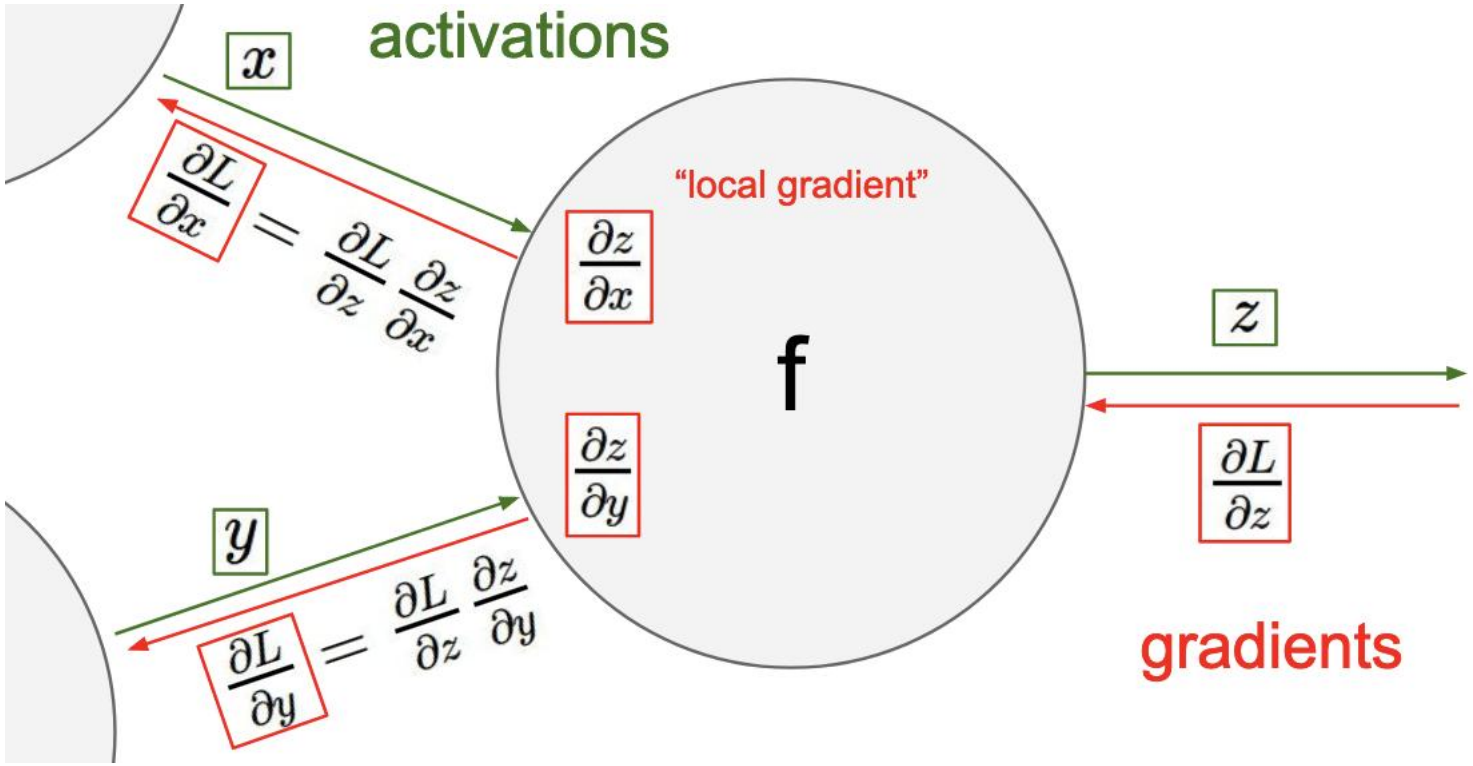
BP local gradient



BP local gradient



BP local gradient

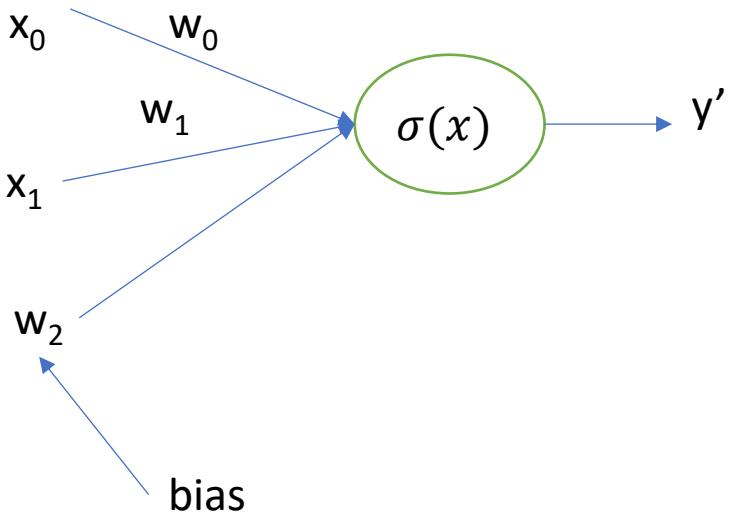


BP local gradient

- Backpropagation is a beautifully local process
  - Every gate in a circuit diagram gets some inputs and Computes two values
    1. its output value and
    2. The *local* gradient of its output with respect to its inputs.
  - Gates can do this completely independently
  - Gates will eventually learn about the gradient of its output value on the final output of the entire circuit

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



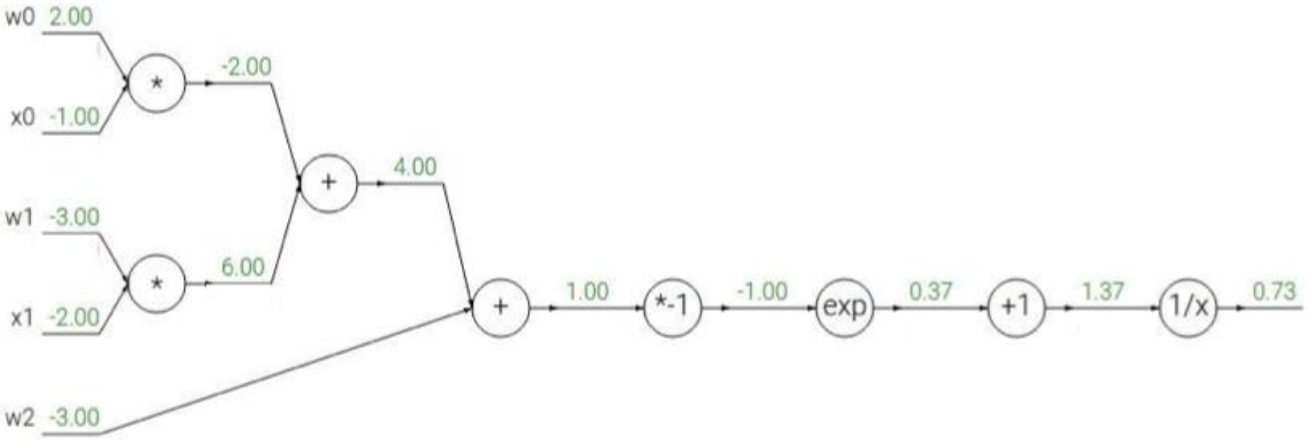
Basic derivative formula:

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

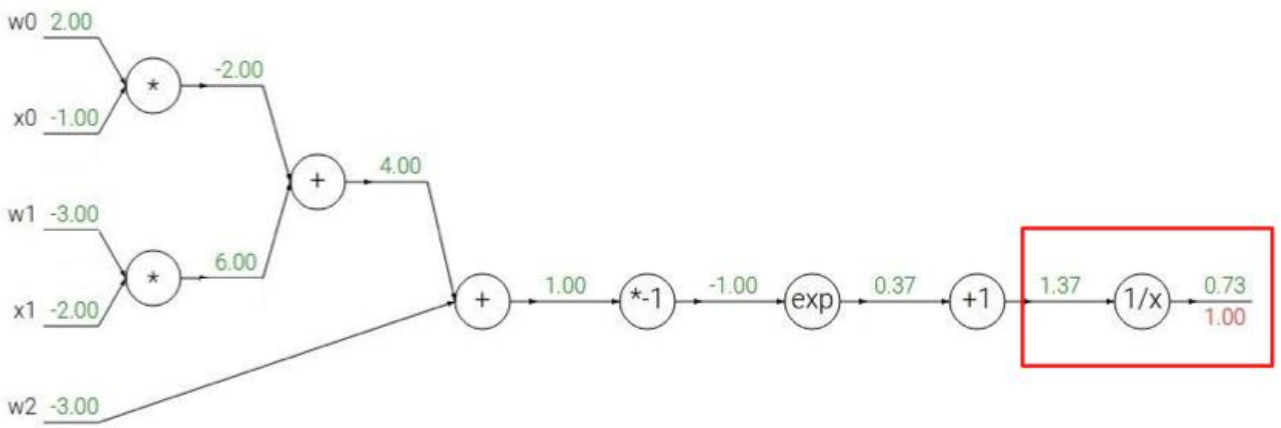
$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$



BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

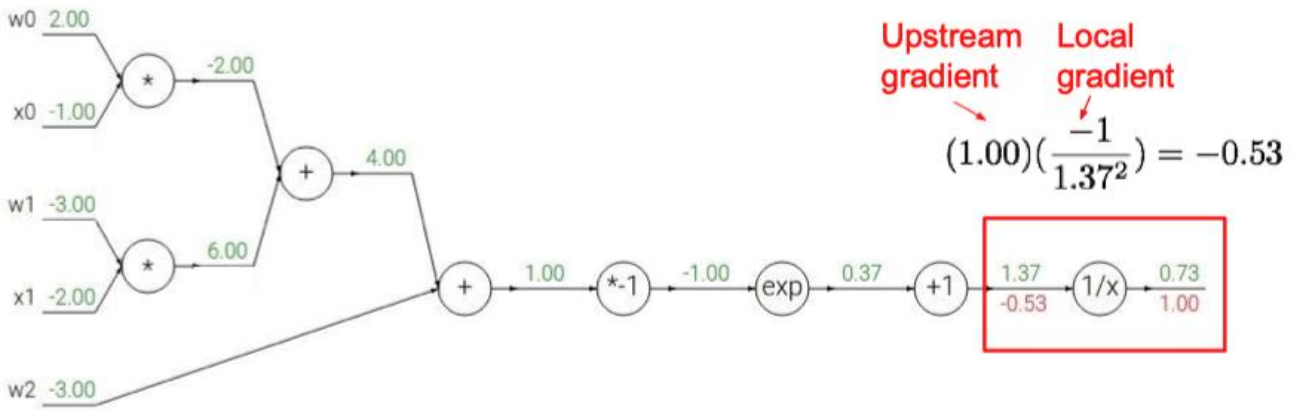
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

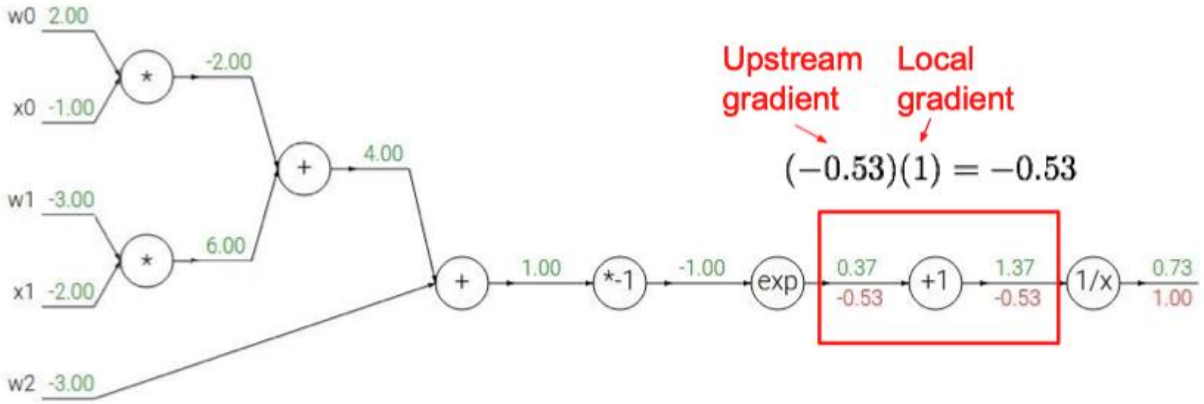
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

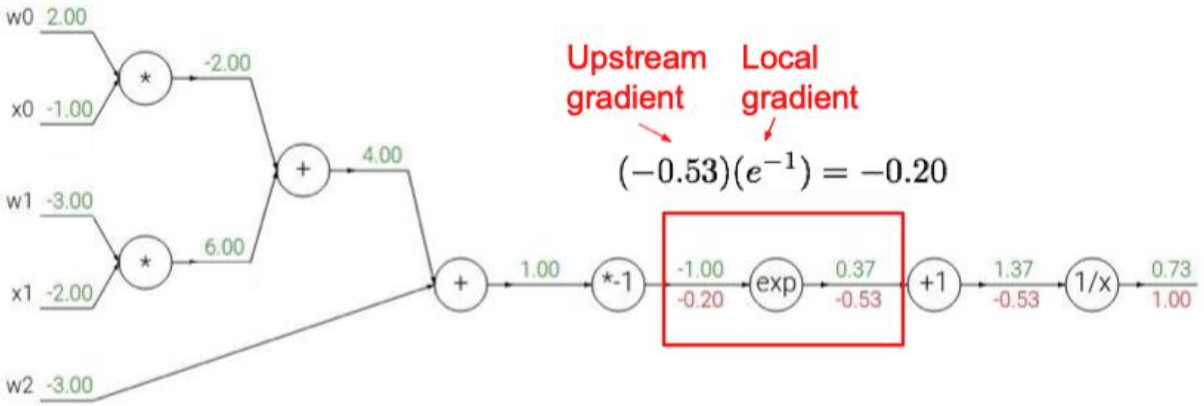
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

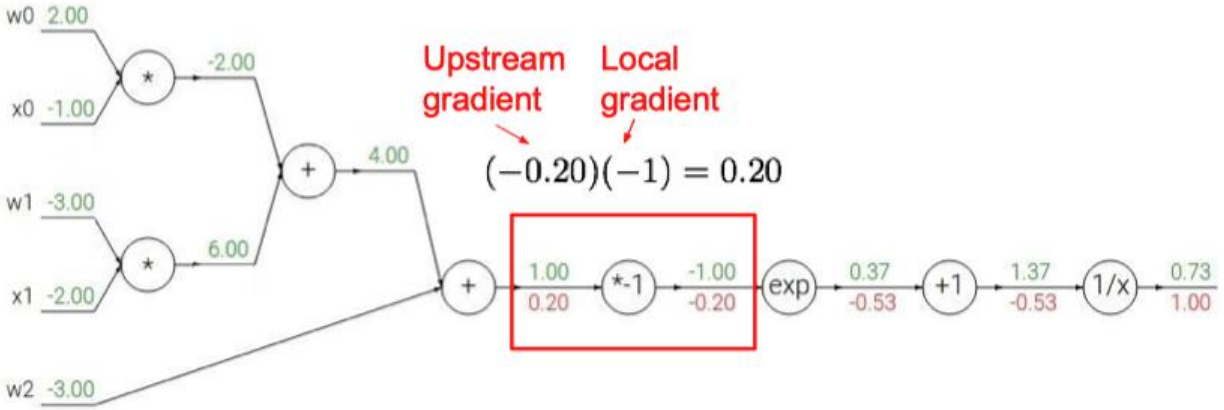
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

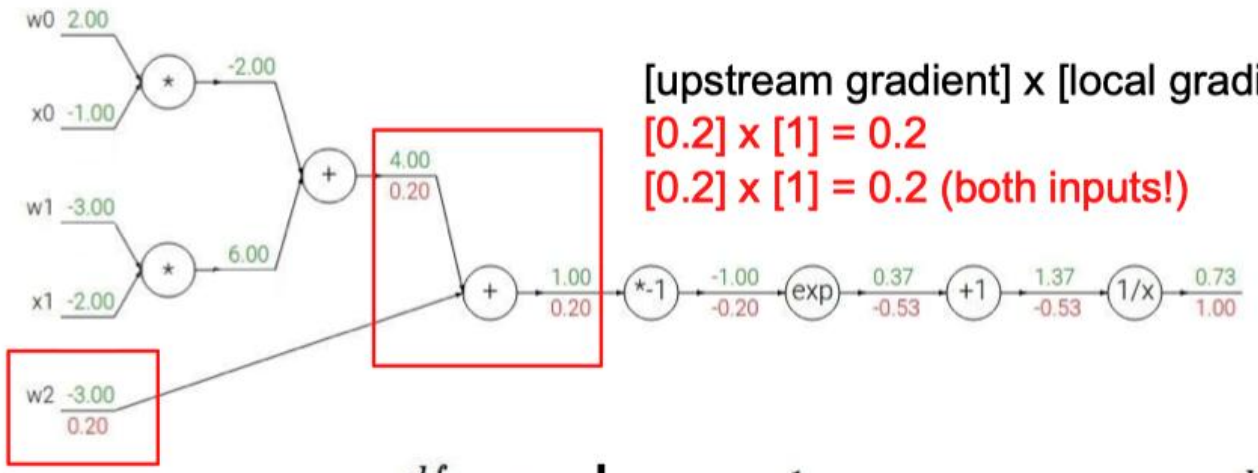
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

[0.2] x [1] = 0.2

[0.2] x [1] = 0.2 (both inputs!)

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

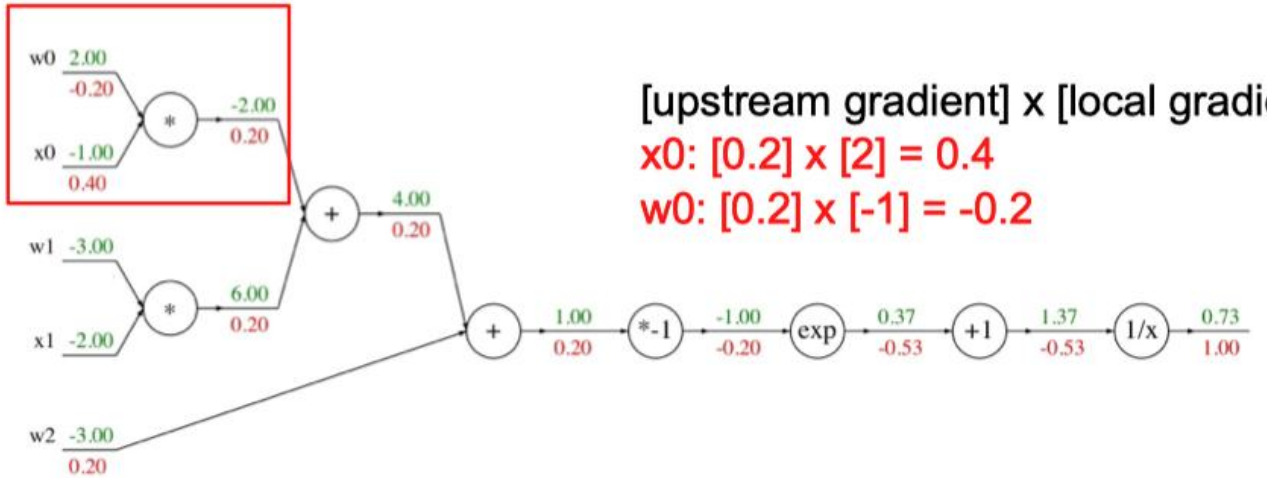
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

BP: sigmoid example

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

x0: [0.2] x [2] = 0.4

w0: [0.2] x [-1] = -0.2

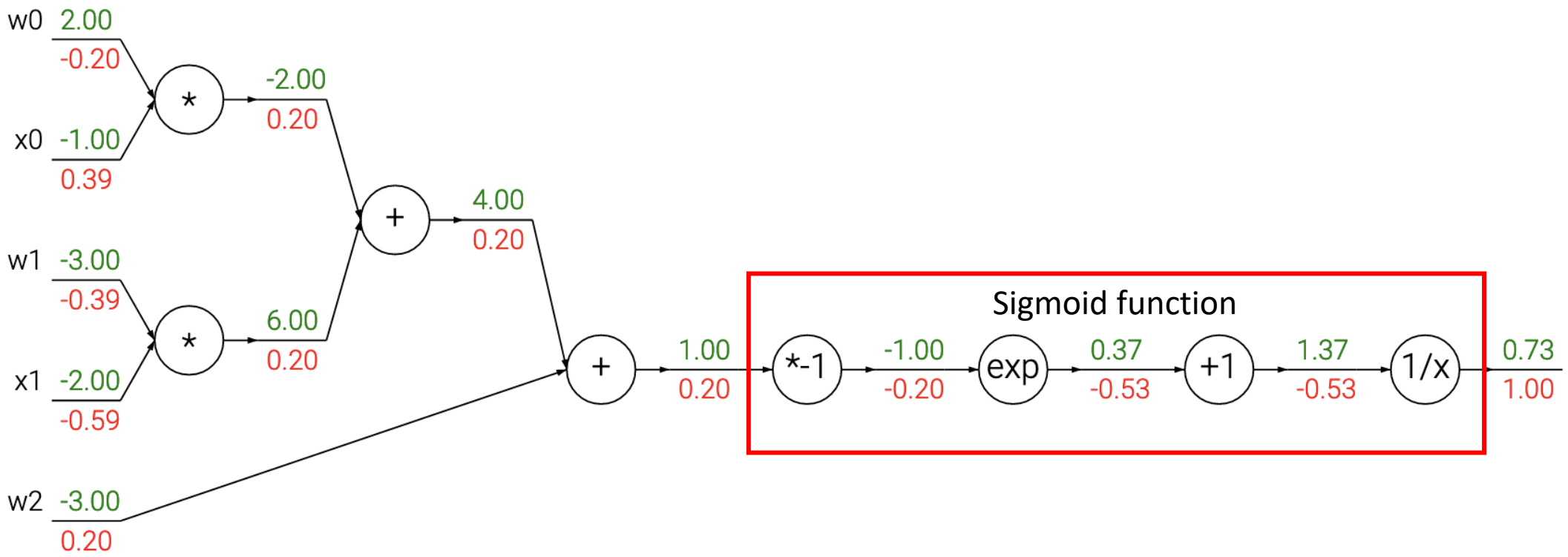
$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

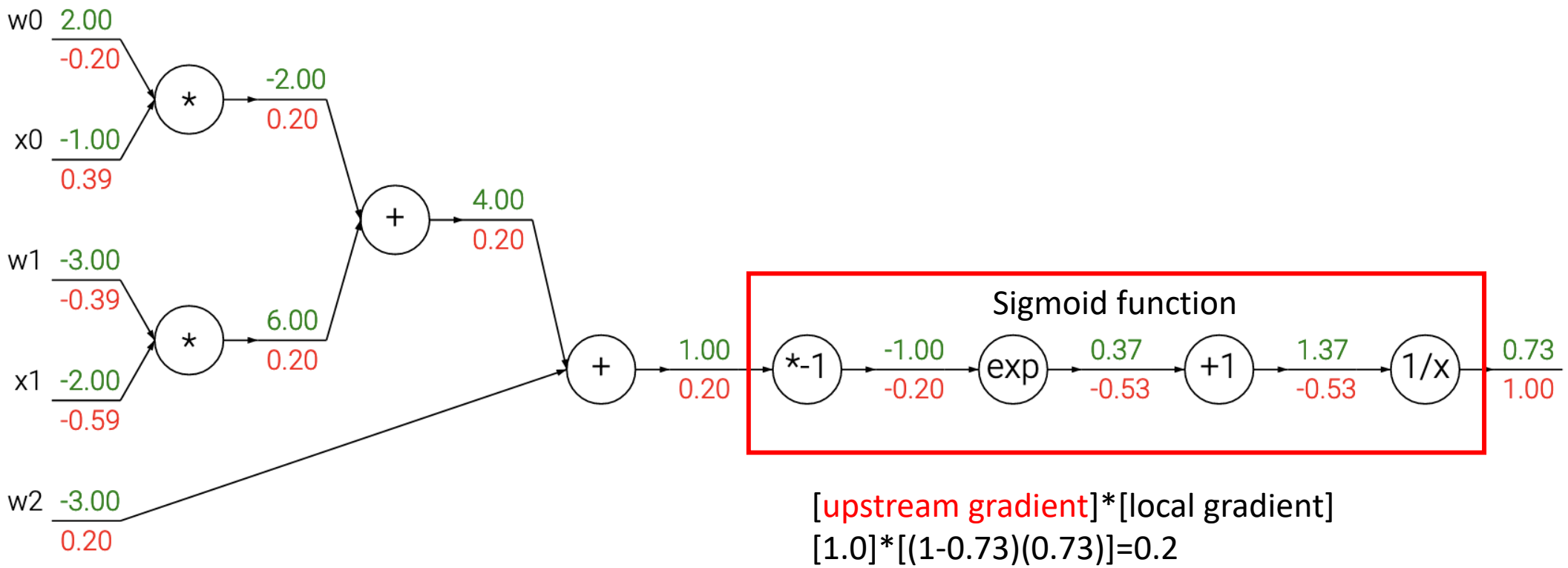
BP: sigmoid example



Sigmoid local gradient: $\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = \left(1-\sigma(x)\right)\sigma(x)$

BP: sigmoid example

- Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

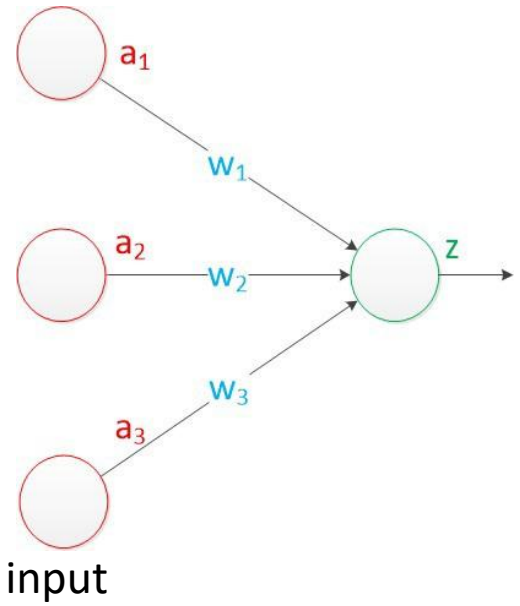


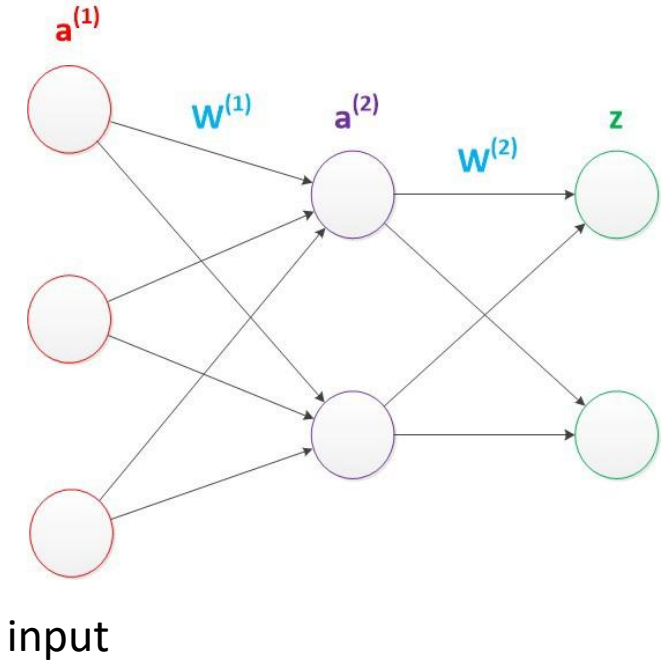
[upstream gradient]*[local gradient]
[1.0]*[(1-0.73)(0.73)]=0.2

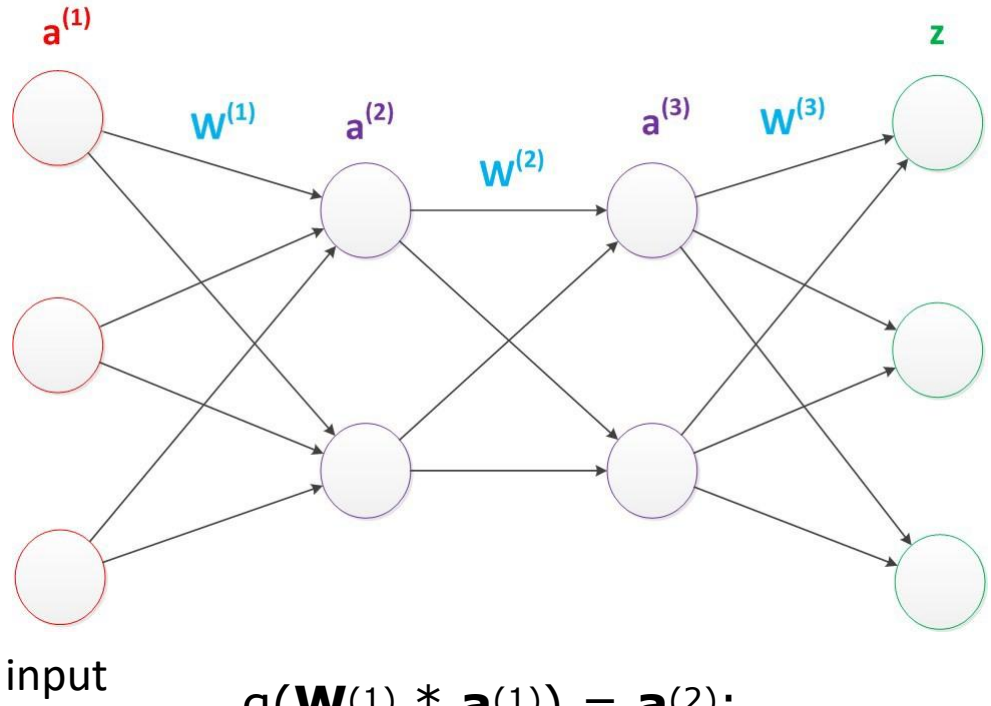
Sigmoid local gradient: $\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = \left(1-\sigma(x)\right)\sigma(x)$

Total Number of Parameters

Multi-layer neural network (deep learning)



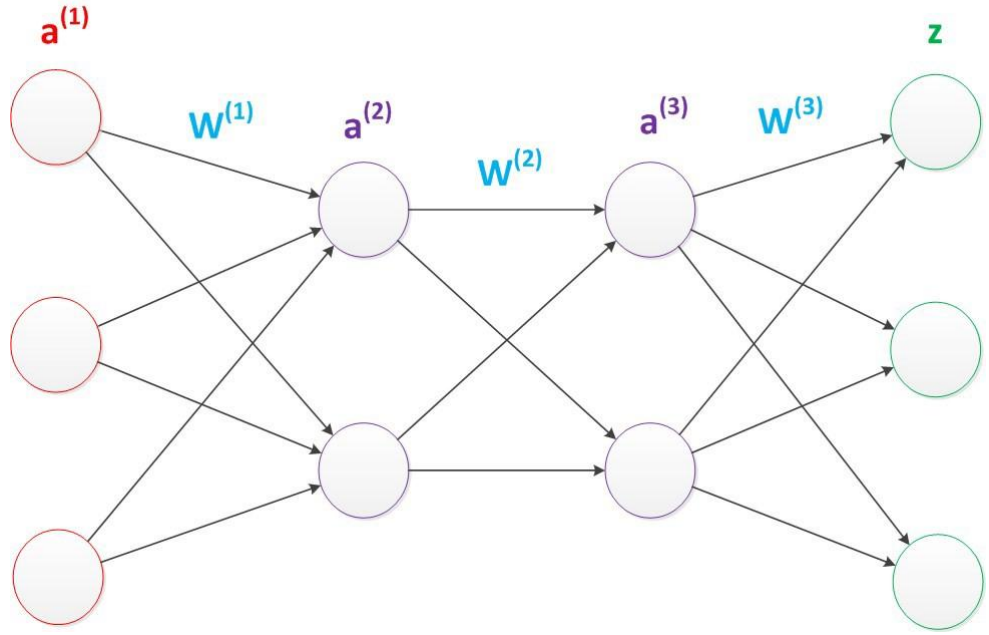




$$g(\mathbf{W}^{(1)} * \mathbf{a}^{(1)}) = \mathbf{a}^{(2)};$$
$$g(\mathbf{W}^{(2)} * \mathbf{a}^{(2)}) = \mathbf{a}^{(3)};$$
$$g(\mathbf{W}^{(3)} * \mathbf{a}^{(3)}) = \mathbf{z};$$

Multi-layer neural network (deep learning)

- There are 6 parameters in W(1), 4 parameters in W(2), and 6 parameters in W(3)

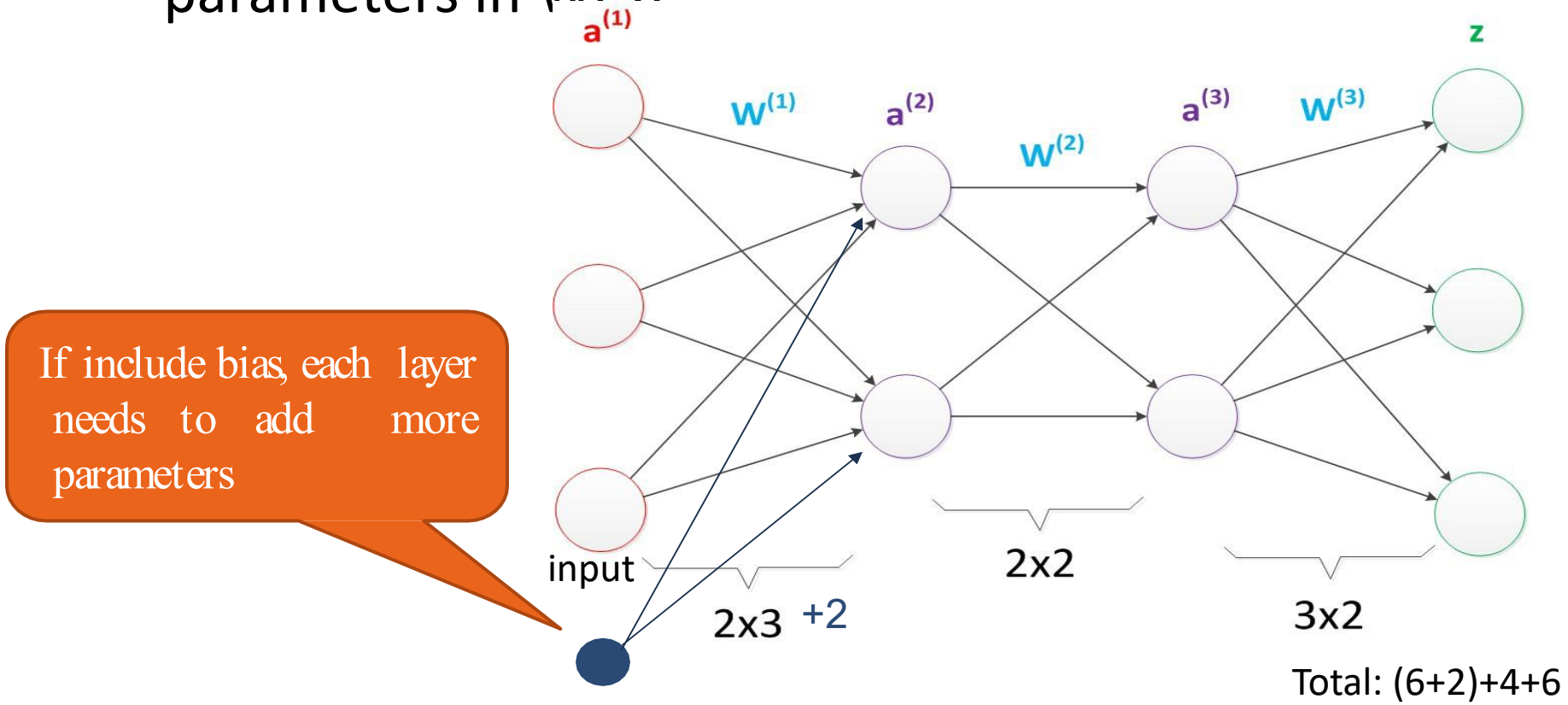


input

Total: 6+4+6

Multi-layer neural network (deep learning)

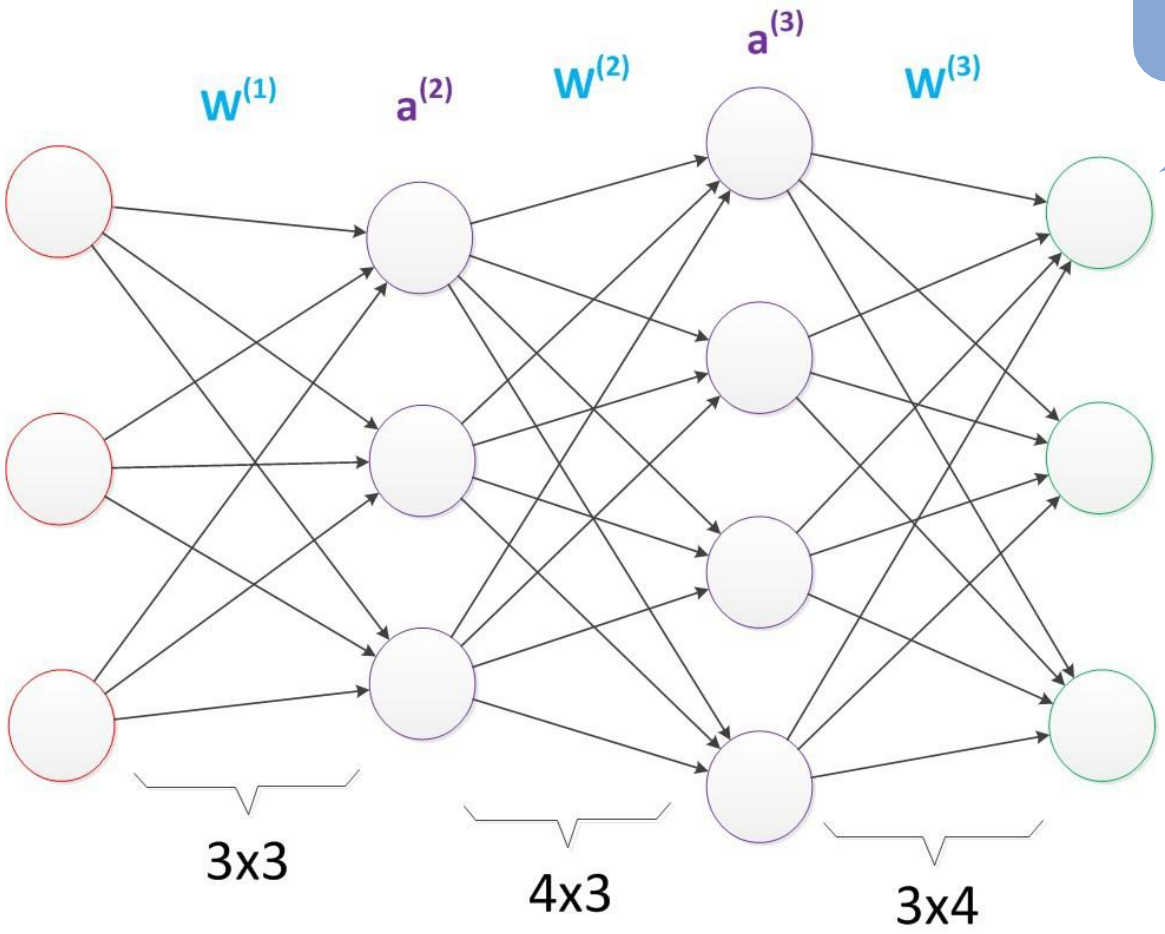
- There are 6 parameters in W(1), 4 parameters in W(2), and 6 parameters in W(3)



Multi-layer neural network (deep learning)

- How many parameters in this network?

The number of parameters of the second neural network is nearly twice that of the first neural network, resulting in better representation capabilities



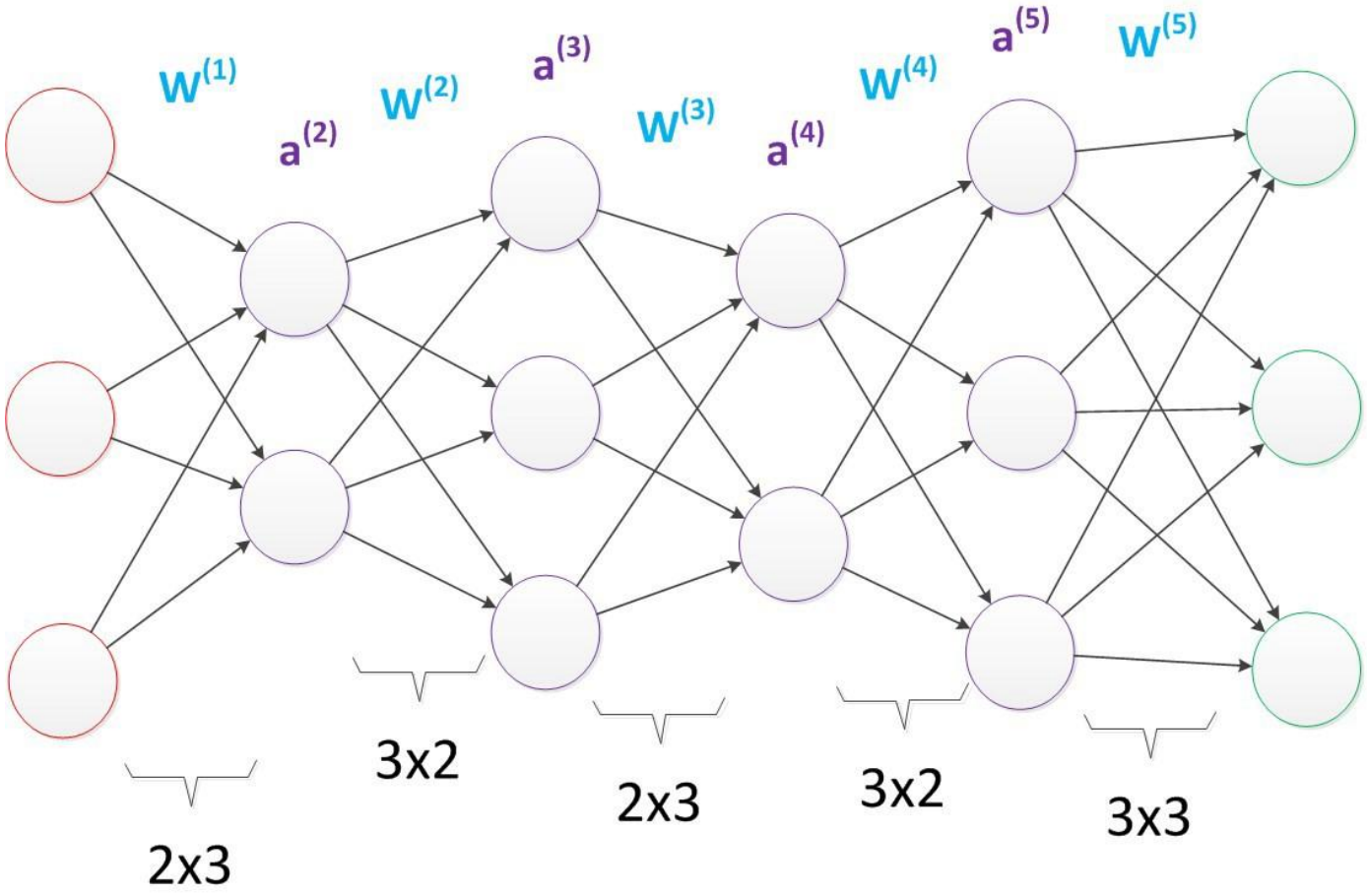
9+12+12=33

Multi-layer neural network (deep learning)

- How many parameters in this network?

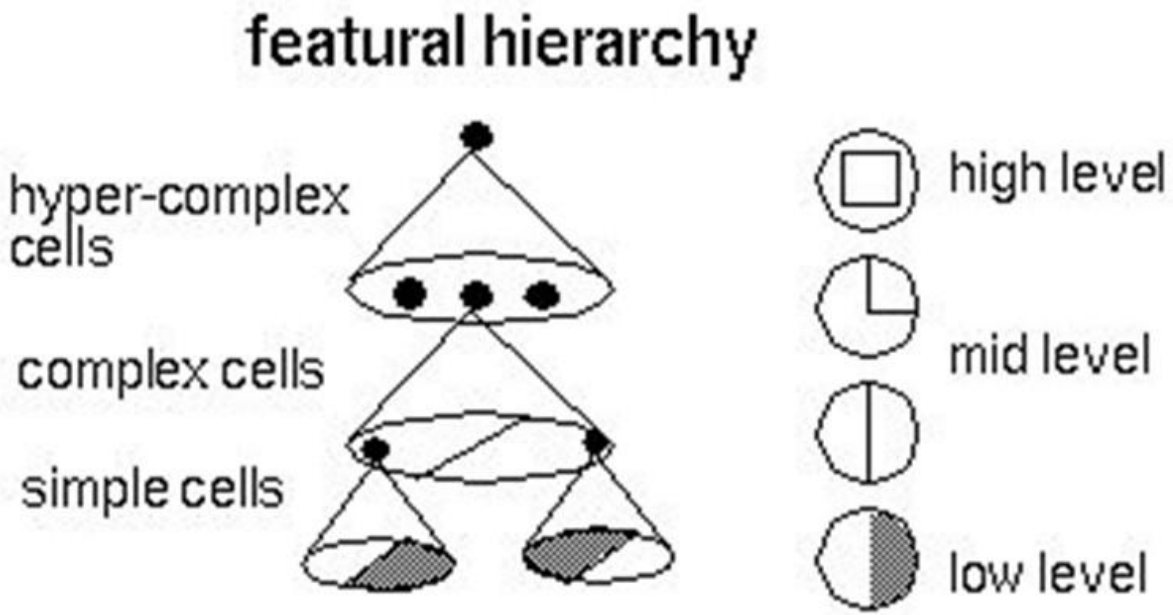
The same number of parameters can be expressed at a deeper level

better representation capabilities



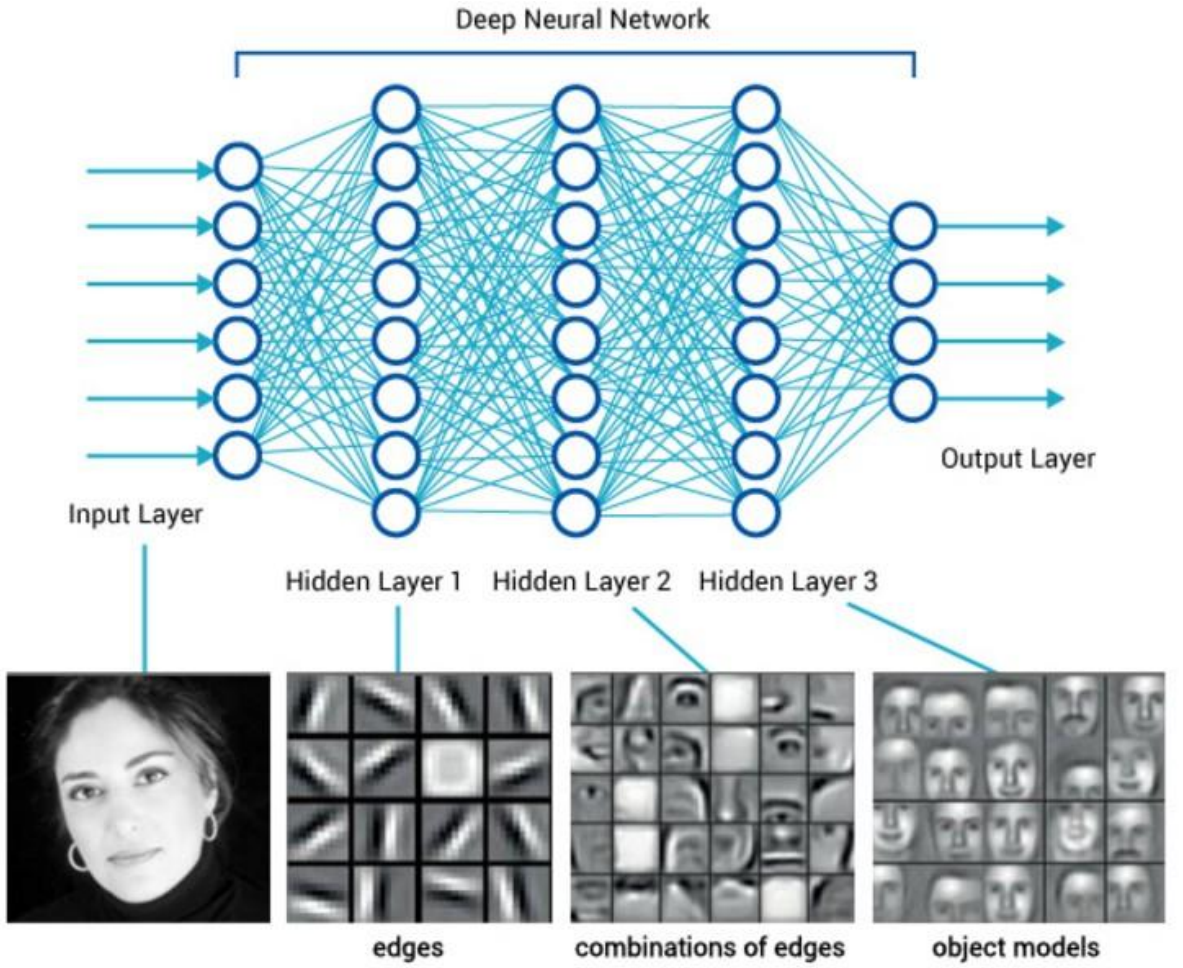
6+6+6+6+9=33

What are the benefits of adding more levels/layers?



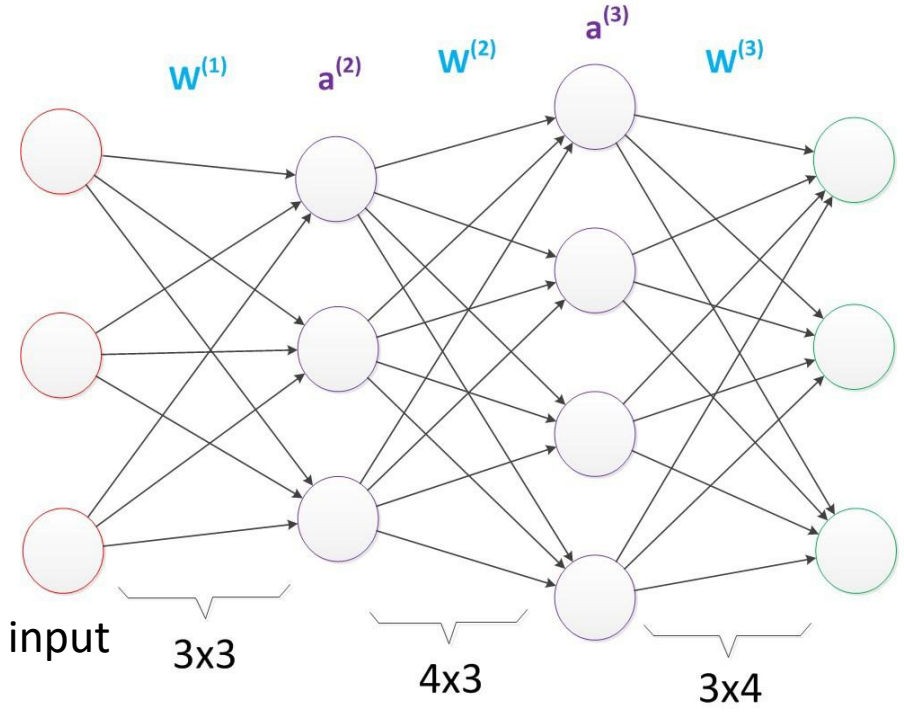
- More in-depth representation features, and stronger function simulation capabilities
- As the number of layers in the network increases, each layer has a deeper abstraction of the previous level
  - 1st layer: margin
  - 2nd layer: shape
  - 3rd layer: pattern
  - ...
  - nth layer: target

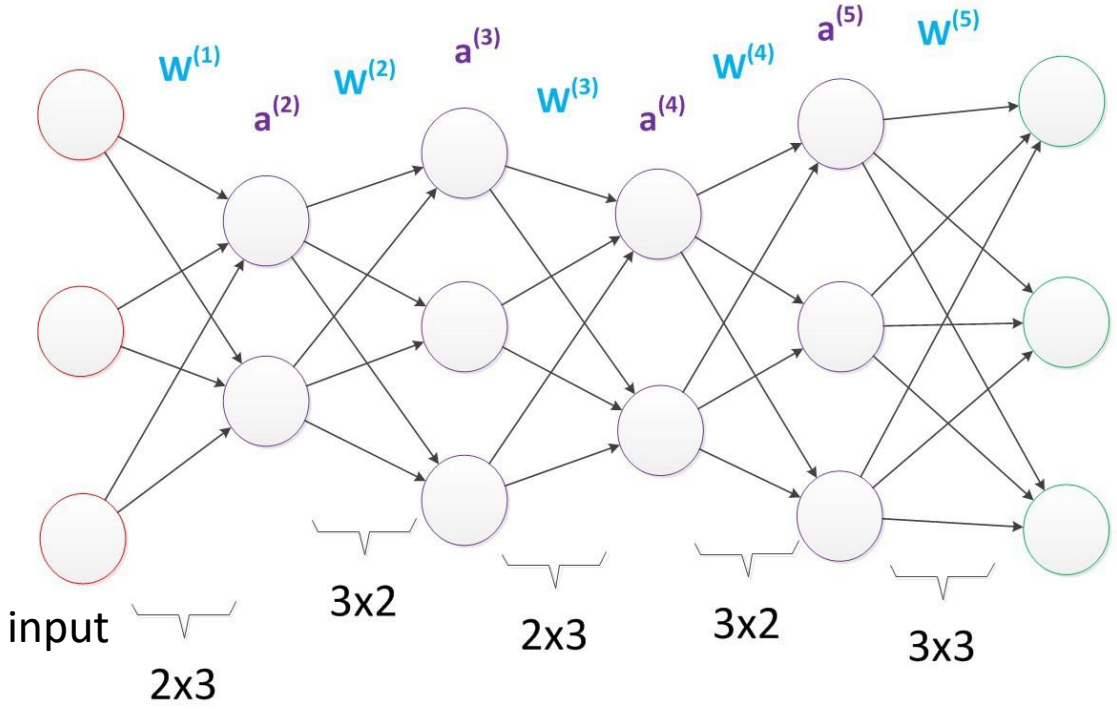
What are the benefits of adding more levels/layers?



Multi-layer neural network (deep learning)

- Research found that in the case of the same number of parameters, deeper networks tend to have better recognition efficiency than shallow networks





Advantages and disadvantages of MLP

- Advantages
  - Capability to learn non-linear models.
  - Capability to learn models in real-time (on-line learning)
- Disadvantages
  - MLP with hidden layers have a non-convex loss function where there exists more than one local minimum. Therefore, different random weight initializations can lead to different validation accuracy.
  - MLP requires tuning a number of hyperparameters such as the number of hidden neurons, layers, and iterations.
  - MLP is sensitive to feature scaling, so it is highly recommended to scale your data.

Summary

- Neural Network Introduction
- Activation Functions
- Multilayer Neural network